

Planeación de acciones para robots de servicio
mediante modelos de lenguaje y sistemas basados en
reglas

Planeación de acciones para robots de servicio a
partir de instrucciones en lenguaje natural mediante
Qwen, CLIPS y modelos GPT

Luis Sergio Cano Olguin

Tutor: Dr. Jesús Savage Carmona

Agradecimientos

Resumen

Índice general

1. Introducción	1
1.1. Contexto y Motivación	2
1.2. Planteamiento del problema	3
1.3. Hipótesis	4
1.4. Objetivos	5
1.5. Estructura del documento	6
2. Antecedentes y Estado del Arte	8
2.1. Sistemas Basados en Reglas	8
2.2. El Lenguaje CLIPS: Características, Ventajas y Limitaciones	9
2.2.1. Características Fundamentales de CLIPS	10
2.3. Modelos de lenguaje para interpretación de instrucciones robóticas	11
2.3.1. Familia de modelos Qwen	12
2.3.2. Modelos GPT	13
2.3.3. Interpretación semántica y generación de planes robóticos	14
2.3.4. Limitaciones de los modelos de lenguaje en robótica	15
2.3.5. Salidas estructuradas y validación	15
2.3.6. Adaptación eficiente de modelos: LoRA y QLoRA	16
2.4. Arquitecturas neuro-simbólicas	16
2.5. Síntesis y conexión con la propuesta	19
3. Formalización del problema y arquitectura conceptual	21
3.1. Contexto arquitectónico y alcance del sistema	22
3.1.1. Integración dentro de la arquitectura ViRBot	22
3.2. GPSR como dominio de referencia	23
3.2.1. La prueba General Purpose Service Robot	23

3.2.2.	Generación de comandos y conocimiento del dominio	24
3.2.3.	Dominio de referencia y alcance lingüístico	26
3.3.	Formalización de órdenes, objetivos y planes	27
3.3.1.	Orden en lenguaje natural	28
3.3.2.	Equivalencia semántica entre formulaciones	29
3.3.3.	Entidades y conocimiento del dominio	29
3.3.4.	Representación canónica de objetivos	31
3.3.5.	Objetivos semánticos y acciones ejecutables	33
3.3.6.	Dependencias y contexto entre objetivos	34
3.4.	Arquitecturas conceptuales evaluadas	37
3.4.1.	Base común de las arquitecturas evaluadas	37
3.4.2.	Arquitectura Qwen-CLIPS	38
3.4.3.	Arquitectura de planificación directa con GPT	40
3.4.4.	Comparación conceptual de las arquitecturas	43
3.5.	Criterios conceptuales de validez	44
3.5.1.	Niveles de validez	45
3.5.2.	Validez de la interpretación semántica	45
3.5.3.	Validez y consistencia del plan	46
3.5.4.	Validez de la respuesta del sistema	47
3.5.5.	Condición de control para la etapa simbólica	47
3.5.6.	Planificación y ejecución física	48
4.	Metodología y diseño de las arquitecturas evaluadas	50
4.1.	Diseño metodológico de la investigación	50
4.1.1.	Problema experimental	50
4.1.2.	Variables y condiciones de comparación	51
4.1.3.	Estrategia general de evaluación	52
4.2.	Construcción del dominio experimental	52
4.2.1.	Fuentes de conocimiento GPSR	52
4.2.2.	Catálogos de personas, objetos y ubicaciones	53
4.2.3.	Familias de tareas	53
4.2.4.	Representación de los objetivos de referencia	54
4.3.	Construcción del conjunto de datos	55
4.3.1.	Adaptación del CommandGenerator	55
4.3.2.	Generación de órdenes	56

4.3.3.	Generación de objetivos canónicos	57
4.3.4.	Variación lingüística	58
4.3.5.	Balance y deduplicación	58
4.3.6.	División de entrenamiento y evaluación	59
4.3.7.	Validación previa del conjunto de datos	59
4.4.	Diseño de la arquitectura Qwen-CLIPS	61
4.4.1.	Función asignada a Qwen	61
4.4.2.	Especialización para interpretación semántica	61
4.4.3.	Interfaz entre objetivos y planificación simbólica	62
4.4.4.	Función asignada a CLIPS	63
4.5.	Diseño de la arquitectura GPT directo	63
4.5.1.	Funciones asignadas al modelo	63
4.5.2.	Conocimiento y restricciones proporcionadas	64
4.5.3.	Contrato de salida	64
4.5.4.	Plan ejecutable, aclaración y rechazo	65
4.6.	Diseño de la comparación	65
4.6.1.	Igualdad del dominio de evaluación	65
4.6.2.	Igualdad del repertorio de acciones	66
4.6.3.	Separación entre interpretación y planificación	66
5.	Implementación	68
5.1.	Plataforma experimental	69
5.1.1.	Robot Justina	69
5.1.2.	Plataforma de cómputo	69
5.1.3.	Entorno de software y ROS 2	70
5.2.	Implementación del conjunto de datos	70
5.2.1.	Generador de órdenes	70
5.2.2.	Catálogos y CompetitionTemplate	71
5.2.3.	Validadores	72
5.2.4.	Dataset final	72
5.3.	Entrenamiento e inferencia de Qwen2.5-7B	73
5.3.1.	Preparación del modelo	73
5.3.2.	Configuración QLoRA	74
5.3.3.	Hiperparámetros de entrenamiento	74
5.3.4.	Selección del checkpoint	75

5.3.5.	Inferencia	76
5.3.6.	Despliegue del modelo	77
5.4.	Implementación del planificador CLIPS	78
5.4.1.	Conversión de objetivos a hechos gpsr-goal	78
5.4.2.	Representación del estado simbólico	80
5.4.3.	Reglas de expansión	81
5.4.4.	Generación de acciones	82
5.4.5.	Condición de finalización	83
5.5.	Integración Qwen-CLIPS con Justina	84
5.5.1.	Componentes ROS 2	84
5.5.2.	Flujo de comunicación	84
5.5.3.	Comunicación con el ejecutor	85
5.6.	Implementación del planificador GPT	86
5.6.1.	Interfaz con la API	86
5.6.2.	System prompt	86
5.6.3.	JSON Schema y Structured Outputs	87
5.6.4.	Generación de objetivos y planes	87
5.6.5.	Tratamiento de aclaraciones y rechazos	88
5.6.6.	Validación determinista	88
5.7.	Reproducibilidad	89
5.7.1.	Versiones de modelos	89
5.7.2.	Configuraciones experimentales	89
5.7.3.	Registro de ejecuciones	89
6.	Validación y experimentación	91
6.1.	Diseño experimental	92
6.1.1.	Conjunto de prueba	92
6.1.2.	Procedimiento experimental	92
6.1.3.	Repeticiones y control de variabilidad	92
6.2.	Condiciones experimentales	92
6.2.1.	Arquitectura Qwen-CLIPS	92
6.2.2.	Arquitectura GPT directo	92
6.2.3.	Condición de control con objetivos de referencia y CLIPS	92
6.3.	Evaluación de la interpretación semántica	92
6.3.1.	Exactitud de objetivos	92

6.3.2.	Validez estructural	92
6.3.3.	Exactitud de entidades y parámetros	92
6.3.4.	Dependencias entre objetivos	92
6.4.	Evaluación de la planificación	92
6.4.1.	Acciones válidas	92
6.4.2.	Consistencia del plan	92
6.4.3.	Cumplimiento de subtareas	92
6.4.4.	Precondiciones y dependencias	92
6.4.5.	Planificabilidad	92
6.5.	Clasificación de errores	92
6.5.1.	Errores de interpretación	92
6.5.2.	Errores de planificación	92
6.5.3.	Errores combinados	92
6.6.	Evaluación física con Justina	92
6.6.1.	Selección de tareas	92
6.6.2.	Procedimiento de ejecución	92
6.6.3.	Criterios de éxito y fallo	92
6.7.	Rendimiento computacional	92
6.7.1.	Latencia	92
6.7.2.	Uso de recursos	92
6.7.3.	Tokens y coste del modelo GPT	92
7.	Resultados	93
7.1.	Resultados de la especialización de Qwen	94
7.1.1.	Comportamiento durante entrenamiento	94
7.1.2.	Evaluación del modelo seleccionado	94
7.2.	Resultados de interpretación semántica	94
7.2.1.	Qwen	94
7.2.2.	GPT	94
7.2.3.	Comparación	94
7.3.	Resultados de planificación	94
7.3.1.	Qwen–CLIPS	94
7.3.2.	GPT directo	94
7.3.3.	Condición de control CLIPS	94
7.3.4.	Comparación	94

7.4.	Análisis de errores	94
7.4.1.	Errores de interpretación	94
7.4.2.	Errores de planificación	94
7.4.3.	Casos representativos	94
7.5.	Resultados de ejecución física	94
7.6.	Rendimiento computacional	94
7.6.1.	Latencia	94
7.6.2.	Recursos computacionales	94
7.6.3.	Tokens y coste	94
8.	Discusión	95
8.1.	Interpretación de los resultados respecto a la hipótesis	96
8.2.	Interpretación semántica	96
8.2.1.	Especialización de Qwen	96
8.2.2.	Capacidad de GPT	96
8.2.3.	Efecto de la variabilidad lingüística	96
8.3.	Planificación de acciones	96
8.3.1.	Expansión determinista mediante CLIPS	96
8.3.2.	Planificación generativa mediante GPT	96
8.3.3.	Separación entre errores de interpretación y planificación	96
8.4.	Comparación Qwen-CLIPS frente a GPT directo	96
8.4.1.	Exactitud	96
8.4.2.	Consistencia	96
8.4.3.	Latencia	96
8.4.4.	Recursos y coste	96
8.4.5.	Dependencia de infraestructura externa	96
8.5.	Generalización fuera de las plantillas GPSR	96
8.6.	Limitaciones y amenazas a la validez	96
8.6.1.	Dominio de tareas	96
8.6.2.	Dataset	96
8.6.3.	Modelo Qwen	96
8.6.4.	Modelo GPT	96
8.6.5.	Plataforma física	96
8.7.	Implicaciones para la planificación en robots de servicio	96

9. Conclusiones y trabajo futuro	97
9.1. Conclusiones principales	97
9.2. Cumplimiento de los objetivos	97
9.3. Evaluación de la hipótesis	97
9.4. Contribuciones de la investigación	97
9.4.1. Representación canónica de órdenes	97
9.4.2. Arquitectura Qwen–CLIPS	97
9.4.3. Comparación con planificación directa mediante GPT	97
9.4.4. Metodología para separar errores de interpretación y planificación	97
9.5. Trabajo futuro	97
10. Conclusiones y Trabajo Futuro	98
10.1. Conclusiones Principales	98
10.2. Propuestas de Trabajo Futuro	98

Índice de figuras

3.1. Arquitectura general ViRBot del robot Justina.	22
3.2. Arquitectura conceptual Qwen-CLIPS.	39
3.3. Arquitectura conceptual del planificador GPT directo.	41

Capítulo 1

Introducción

Los robots de servicio representan un campo de proyección para la inteligencia artificial y la automatización, con aplicaciones que van desde la asistencia a personas mayores hasta la gestión autónoma de entornos residenciales (1), donde deben interpretar instrucciones expresadas por personas y convertirlas en acciones que puedan ejecutarse mediante módulos de navegación, percepción, manipulación e interacción.(2).

La transformación de una instrucción en lenguaje natural a un plan robótico presenta dos problemas relacionados. El primero es semántico: identificar correctamente los objetivos, entidades, destinos y relaciones expresados por el usuario. El segundo es simbólico: convertir dicha interpretación en una secuencia de acciones pertenecientes al repertorio real del robot y con parámetros compatibles con el ejecutor.

Los sistemas basados en reglas, han demostrado ser robustos y predecibles en escenarios estructurados, como líneas de producción o laboratorios controlados (3). En particular, CLIPS utiliza hechos y reglas de producción para determinar qué transformaciones deben aplicarse a partir de un estado simbólico. Sin embargo, su rigidez los hace poco adaptables ante variaciones no previstas en el entorno o ante comandos expresados en lenguaje natural con alto grado de ambigüedad o complejidad (4).

Recientemente, los modelos de lenguaje grande (LLMs), como ChatGPT, han emergido como herramientas capaces de interpretar y generar lenguaje natural, e incluso de actuar como agentes de planificación autónomos (5). Su flexibilidad contextual permite traducir instrucciones verbales en secuencias de acciones, complementando así la solidez de los sistemas basados en reglas. No obstante, su naturaleza probabilística puede producir formatos inválidos, entidades inexistentes o acciones que no pertenecen a las capacidades del robot. Por lo tanto la falta de garantías formales de seguridad limitan

su uso directo en aplicaciones robóticas donde la integridad física y la predictibilidad son prioritarias (6).

Esta tesis estudia dos aproximaciones para resolver el problema. La primera es una arquitectura neuro-simbólica en la que un modelo Qwen2.5-7B, ajustado mediante QLoRA, transforma una orden en objetivos canónicos representados en JSON. Posteriormente, un módulo determinista normaliza y convierte esos objetivos en hechos, que son procesados por reglas de producción en CLIPS para generar el plan compatible con el ejecutor del robot. La segunda aproximación emplea un modelo GPT como planificador generativo directo. En este caso, el modelo recibe la orden, el contexto y el catálogo de acciones, produciendo una respuesta estructurada mediante JSON Schema que es revisada por un validador determinista antes de considerarse ejecutable.

Las dos aproximaciones no se combinan mediante un selector de planes durante la ejecución. Se implementan como arquitecturas independientes y se comparan experimentalmente bajo el mismo conjunto de órdenes, contexto y criterios de evaluación. Esta comparación permite analizar el compromiso entre interpretación semántica, consistencia, trazabilidad, flexibilidad, latencia, recursos computacionales y coste de uso.

1.1. Contexto y Motivación

El área de la robotica orientada al servicio doméstico ha evolucionado significativamente en la última década, impulsada por avances en percepción, planificación y interacción humano-robot (1).

El robot de servicio Justina, desarrollado en el Laboratorio de Bio-Robótica de la UNAM, integra módulos de percepción, navegación, manipulación e interacción mediante ROS 2. Para ejecutar una orden, el robot necesita una representación intermedia que conecte el lenguaje del usuario con las acciones ofrecidas por dichos módulos.

Estos sistemas se han diseñado para operar en entornos no estructurados como hogares, hospitales o centros de cuidado, donde deben realizar tareas que van desde la entrega de objetos hasta la asistencia en actividades de la vida diaria. En este contexto, la capacidad de planificar acciones de manera autónoma, segura y adaptativa se convierte en un requisito fundamental.

La planificación de acciones en robótica ha sido tradicionalmente abordada mediante sistemas basados en reglas (*Rule-Based Systems, RBS*), que ofrecen un marco predecible y verificable para la toma de decisiones (2). Sin embargo, una solución exclusivamente

simbólica puede generar planes consistentes cuando recibe objetivos correctamente estructurados, pero no resuelve directamente la interpretación del lenguaje natural. En contraste, un modelo de lenguaje puede interpretar distintas formulaciones de una orden, aunque su salida no es necesariamente compatible con las restricciones del robot. La motivación de este trabajo es evaluar si la separación entre interpretación neuronal y expansión simbólica permite obtener planes ejecutables y trazables, y contrastar sus resultados con los de un planificador basado exclusivamente en un modelo GPT.

El interés de la comparación no consiste en demostrar que un paradigma es universalmente superior. Se busca identificar que arquitectura presenta ventajas dependiendo del comando, así como que errores introduce cada etapa y cuáles son los costes asociados con su despliegue local o mediante una API externa.

1.2. Planteamiento del problema

Una orden puede contener variaciones lingüísticas, referencias espaciales, objetos, personas, destinos y varias subtarefas relacionadas. Para que el robot pueda ejecutarla, la orden debe convertirse en una secuencia válida de acciones primitivas, para esto el sistema basados en reglas CLIPS, fundamentado en la lógica simbólica y el ciclo *reconocer-actuar* (3), usa un conjunto de hechos y reglas bien definidos para producir una respuesta determinista y explicable, lo que lo hace idóneo para aplicaciones donde la seguridad es crítica, como entornos industriales o médicos.

La conversión por parte de CLIPS enfrenta los siguientes problemas:

1. **Rigidez interpretativa** No se pueden procesar comandos en lenguaje natural complejo o ambiguo.
2. **Falta de adaptabilidad:** Las reglas deben ser definidas a priori; cualquier situación no prevista en la base de conocimiento puede llevar al fracaso o a un comportamiento no deseado.
3. **Dificultad para gestionar la incertidumbre:** No manejan bien la información incompleta o cambiante del entorno en tiempo real.
4. **Escalabilidad limitada:** Mantener y extender un conjunto grande de reglas para cubrir todos los escenarios posibles resulta complejo y laborioso.

Por otro lado, los resultados deben satisfacer al menos las siguientes condiciones (6):

- Las entidades finales deben pertenecer al catálogo o ser tratadas explícitamente como desconocidas.

- Los objetivos deben conservar el significado y el orden causal de la instrucción.
- Las acciones generadas deben pertenecer al repertorio del ejecutor.
- Cada acción debe contener los parámetros requeridos.
- La secuencia debe respetar precondiciones simbólicas básicas, como la ubicación del robot o la posesión de un objeto.

Los modelos de lenguaje pueden ampliar la cobertura lingüística, aunque también pueden generar resultados inexistentes o inconsistentes. El problema de investigación se formula, por tanto, de la siguiente manera:

¿Qué diferencias de exactitud, consistencia, robustez y coste existen entre una arquitectura neuro-simbólica Qwen-CLIPS y un planificador generativo basado en GPT al convertir órdenes GPSR en planes para un robot de servicio?

1.3. Hipótesis

Para la interpretación y planeación de órdenes, una arquitectura neuro-simbólica compuesta por Qwen y CLIPS producirá una mayor proporción de planes estructuralmente válidos, trazables y consistentes que un planificador generativo basado directamente en GPT. Por otra parte, se espera que el planificador GPT directo presente mayor flexibilidad ante formulaciones lingüísticas novedosas, pero con mayor variabilidad, latencia y coste de inferencia mediante API.

Se espera que los sistemas:

- Mejore la tasa de éxito en la ejecución de tareas en entornos domésticos.
- Reduzca el tiempo de respuesta ante comandos complejos.
- Mantenga un comportamiento seguro y predecible en situaciones críticas.
- Permita una interacción más natural e intuitiva con usuarios no expertos.

La hipótesis se evaluará de forma separada para la interpretación de objetivos y para la generación del plan completo. De esta manera será posible determinar si un error proviene del modelo de lenguaje, de la conversión simbólica o de la cobertura de las reglas CLIPS.

1.4. Objetivos

Diseñar, implementar y evaluar una arquitectura neuro-simbólica Qwen–CLIPS para interpretar órdenes y generar planes compatibles con un robot de servicio, comparándola con un planificador generativo basado en GPT con salida estructurada.

■ Objetivos específicos

- Definir una representación canónica de objetivos y su correspondencia con las acciones disponibles en el ejecutor.
- Ajustar Qwen2.5-7B mediante QLoRA para convertir órdenes en objetivos canónicos representados en JSON.
- Implementar la normalización, validación y conversión de los objetivos generados por Qwen en hechos.
- Implementar reglas de producción en CLIPS que expandan objetivos de alto nivel en secuencias de acciones compatibles con el ejecutor.
- Diseñar un planificador GPT directo mediante un prompt, un JSON Schema y un validador determinista.
- Construir un conjunto experimental que incluya órdenes conocidas, paráfrasis, composiciones de objetivos, entradas ambiguas y casos fuera del dominio.
- Comparar las arquitecturas en términos de exactitud semántica, validez estructural, robustez, latencia, consumo de recursos, variabilidad y coste.
- Validar en simulación, robot físico y durante la RoboCup 2026 el resultado los planes generados.

■ Contribuciones esperadas

- Una representación intermedia de objetivos que conecta la interpretación lingüística con la planeación simbólica.
- Un modelo Qwen2.5-7B ajustado mediante QLoRA para traducir órdenes a objetivos canónicos.
- Una integración ROS 2–CLIPS que convierte dichos objetivos en planes compatibles con el ejecutor.
- Un planificador GPT directo restringido mediante prompt, JSON Schema y validación determinista.
- Un protocolo experimental reproducible para comparar planeación neuro-simbólica y planeación generativa.
- Un análisis de errores por etapa que diferencia fallos semánticos, estructurales, simbólicos y de ejecución.

1.5. Estructura del documento

Este documento está organizado en los siguientes capítulos que describen el sistema propuesto para el desarrollo de esta tesis, abarcando el planteamiento, desarrollo, implementación y validación del sistema híbrido de planificación. La estructura es la siguiente:

1. **Capítulo 1: Introducción.** Presenta el contexto, motivación, problemática, hipótesis y objetivos de la investigación, así como la justificación del enfoque híbrido en robótica de servicio doméstico.
2. **Capítulo 2: Antecedentes y estado del arte.** Revisa la evolución de los sistemas basados en reglas en robótica, las características de CLIPS, los enfoques modernos de aprendizaje automático y los sistemas híbridos existentes.
3. **Capítulo 3: Marco teórico y conceptual.** Describe los fundamentos de la planificación jerárquica, los sistemas de producción, los modelos de lenguaje grande (LLMs) y la arquitectura de integración propuesta.
4. **Capítulo 4: Metodología.** Detalla el diseño del sistema híbrido, incluyendo la definición de reglas en CLIPS, la integración con ChatGPT/Qwen, y los mecanismos de planificación dual y validación.
5. **Capítulo 5: Implementación.** Explica los entornos de desarrollo, la configuración técnica, la interfaz con APIs y las pruebas utilizados para validar la integración de los módulos.
6. **Capítulo 6: Escenarios de validación y experimentación.** Presenta el diseño experimental, los escenarios de prueba y las métricas cuantitativas y cualitativas utilizadas para evaluar el sistema.
7. **Capítulo 7: Análisis de resultados.** Compara el rendimiento del sistema híbrido frente al sistema basado solo en reglas, evalúa la efectividad de los LLMs y discute limitaciones y costos computacionales.
8. **Capítulo 8: Discusión.** Interpreta los resultados en relación con los objetivos, analiza ventajas y desventajas del enfoque híbrido, y explora implicaciones éticas y de seguridad.
9. **Capítulo 9: Contribuciones y relevancia.** Destaca la aportación técnica del marco híbrido, su aplicabilidad práctica en distintos sectores y su relevancia académica como benchmark.
10. **Capítulo 10: Conclusiones y trabajo futuro.** Resume los hallazgos principales y propone líneas de investigación futura, como el fine-tuning de LLMs y la

mejora de mecanismos de seguridad.

Capítulo 2

Antecedentes y Estado del Arte

Este capítulo presenta los trabajos y tecnologías relacionados con la transformación de instrucciones de lenguaje natural a planes para robots de servicio. La revisión se concentra en cuatro componentes relevantes para la tesis: sistemas basados en reglas, CLIPS, modelos de lenguaje para interpretación y planificación, y arquitecturas neuro-simbólicas. Los métodos de percepción, navegación o control se mencionan únicamente cuando contribuyen directamente a la interfaz entre la orden y el plan de tareas.

2.1. Sistemas Basados en Reglas

Los sistemas basados en reglas (*Rule-Based Systems, RBS*) constituyen uno de los paradigmas clásicos de la inteligencia artificial (3). Su utilización se consolidó con el desarrollo de los sistemas expertos durante las décadas de 1970 y 1980, en los que el conocimiento de especialistas humanos era representado explícitamente mediante hechos y reglas dentro de dominios bien delimitados (2). En robótica, la adopción de estos sistemas se popularizó en los años 80 y 90, particularmente en entornos industriales estructurados, donde la previsibilidad y seguridad eran prioritarias.

Este tipo de representación resulta útil cuando las condiciones bajo las cuales deben ejecutarse determinadas acciones pueden expresarse de manera explícita. Las reglas permiten relacionar estados, eventos u objetivos con decisiones específicas, facilitando la construcción de comportamientos reproducibles dentro de dominios previamente definidos. Entre las herramientas desarrolladas para implementar este paradigma se encuentra CLIPS (*C Language Integrated Production System*), desarrollado originalmente por la NASA. CLIPS proporciona mecanismos para representar conocimiento mediante

hechos y reglas de producción, junto con un motor de inferencia encargado de determinar qué reglas deben activarse de acuerdo con el estado de la base de conocimiento (7).

En cuanto a aplicaciones, los sistemas basados en reglas han demostrado su utilidad en diversos dominios robóticos:

1. **Robótica industrial:** Cadenas de montaje automatizadas, donde las tareas son repetitivas y el entorno está controlado. Los RBS se utilizan para secuenciar operaciones, gestionar excepciones y garantizar la seguridad de los operarios humanos (1).
2. **Robótica de servicio y doméstica:** En tareas simples como la entrega de objetos, navegación en interiores y asistencia a personas con movilidad reducida.
3. **Robótica espacial y de exploración:** Sistemas como Remote Agent de la NASA emplearon arquitecturas basadas en reglas para la planificación y ejecución autónoma de experimentos en misiones no tripuladas (8).

Su principal ventaja para esta tesis es que permiten establecer una correspondencia explícita entre un objetivo simbólico y las acciones emitidas para el robot. Sin embargo, la cobertura del sistema está limitada por las reglas y los tipos de hechos definidos previamente.

2.2. El Lenguaje CLIPS: Características, Ventajas y Limitaciones

CLIPS es un lenguaje de programación basado en reglas, diseñado específicamente para la construcción de sistemas expertos y aplicaciones basadas en conocimiento (7). Concebido originalmente como una alternativa a los costosos sistemas implementados en (*LISP*, *LISt Processing*) en estaciones de trabajo especializadas, CLIPS destacó por su portabilidad, eficiencia y su capacidad para ejecutarse en hardware de bajo costo (3). Su adopción se extendió rápidamente en la comunidad de robótica e inteligencia artificial, convirtiéndose en una herramienta de referencia para la implementación de sistemas de producción (2).

2.2.1. Características Fundamentales de CLIPS

CLIPS es un entorno para el desarrollo de sistemas basados en conocimiento que ofrece hechos, plantillas, reglas de producción, funciones y un motor de inferencia que permite razonar sobre una base de conocimiento mediante reglas de producción (3).

En el contexto de esta tesis, las propiedades relevantes de CLIPS son:

1. **Encadenamiento hacia adelante (*forward chaining*):** CLIPS utiliza predominantemente encadenamiento hacia adelante. A partir de los hechos presentes en la base de conocimiento, el motor identifica las reglas cuyas condiciones se satisfacen y las incorpora a la agenda de ejecución. La ejecución de estas reglas puede modificar posteriormente la base de hechos y habilitar nuevas reglas (7).
2. **Representación explícita del conocimiento:** Además de reglas, CLIPS permite representar conocimiento mediante hechos ordenados y plantillas (*deftemplate*), así como mediante funciones definidas por el usuario en C (7). Esto facilita la integración con sistemas de percepción y control de bajo nivel (9).
3. **Trazabilidad de las reglas activadas:** Debido a que las decisiones dependen de hechos y reglas explícitas, es posible inspeccionar las reglas que son activadas durante una ejecución. Cuando se mantienen constantes los hechos iniciales, las reglas y la estrategia de resolución de conflictos, el comportamiento del motor puede reproducirse de manera determinista.
4. **Portabilidad:** CLIPS fue implementado en C con el propósito de facilitar su utilización en diferentes plataformas. Esta característica permite incorporar el motor de inferencia dentro de sistemas de software más amplios sin depender de una plataforma especializada (3,7).
5. **Integración con otros lenguajes:** CLIPS proporciona interfaces que permiten incorporarlo dentro de aplicaciones desarrolladas en otros lenguajes. En el caso de Python, esta comunicación puede realizarse mediante bindings externos que exponen las funciones del motor de CLIPS. Esto permite utilizar el sistema basado en reglas como un componente dentro de una arquitectura robótica más amplia.

Estas propiedades contribuyen a restringir y validar el comportamiento del sistema, pero no implican, por sí mismas, una garantía de seguridad ni constituyen un mecanismo de verificación formal.

CLIPS tampoco interpreta directamente lenguaje natural ni aprende reglas a partir de los ejemplos empleados en esta investigación. Para recibir una orden requiere una etapa externa que produzca objetivos estructurados. Además, la integración con ROS

2 no es una capacidad intrínseca de CLIPS, sino una interfaz implementada específicamente para comunicar el motor de reglas con los nodos del robot.

2.3. Modelos de lenguaje para interpretación de instrucciones robóticas

El desarrollo del aprendizaje automático, y en particular del aprendizaje profundo, ha ampliado las capacidades de los sistemas robóticos para procesar información compleja y entradas con un alto grado de variabilidad (10). A diferencia de los sistemas basados en reglas, en los que parte del conocimiento del dominio debe representarse explícitamente, los enfoques de aprendizaje automático permiten obtener representaciones y patrones a partir de datos (2).

En el procesamiento de información secuencial, arquitecturas como las redes neuronales recurrentes (RNN), LSTM y GRU permitieron modelar dependencias entre los elementos de una secuencia y fueron utilizadas en diferentes tareas relacionadas con el procesamiento del lenguaje natural (10).

Posteriormente, la arquitectura Transformer introdujo un enfoque basado principalmente en mecanismos de atención, permitiendo modelar relaciones entre los elementos de una secuencia sin depender exclusivamente de recurrencia (5). Esta arquitectura constituye la base de gran parte de los modelos de lenguaje de gran escala desarrollados posteriormente.

Los *Large Language Models* (LLMs) basados en Transformers pueden procesar y generar lenguaje natural utilizando el contexto de una instrucción para producir diferentes tipos de representaciones. En robótica, estos modelos se han utilizado para interpretar comandos complejos, traducir lenguaje natural a representaciones simbólicas e incluso generar planes a partir de descripciones de tareas de alto nivel (6).

Estas capacidades resultan relevantes para esta investigación debido a que permiten utilizar un modelo de lenguaje como interfaz entre las expresiones proporcionadas por el usuario y las representaciones estructuradas utilizadas posteriormente por los componentes de planificación.

2.3.1. Familia de modelos Qwen

Qwen es una familia de modelos de lenguaje desarrollada por el equipo Qwen de Alibaba Cloud. La familia incluye modelos de diferentes tamaños y variantes destinadas a tareas generales y al seguimiento de instrucciones (11). Estos modelos están basados en la arquitectura Transformer y pueden utilizarse en tareas como generación de texto, respuesta a preguntas, extracción de información y producción de contenido estructurado.

Qwen2.5 corresponde a una generación de esta familia que incluye modelos con diferentes números de parámetros. Para esta investigación se utiliza **Qwen/Qwen2.5-7B** como componente neuronal encargado de la interpretación de las instrucciones proporcionadas al robot. Los criterios que motivaron la selección de esta variante se desarrollan posteriormente.

En la arquitectura propuesta, Qwen2.5-7B funciona como intérprete semántico y no como generador del plan final. El modelo recibe una orden expresada en lenguaje natural y produce una secuencia de objetivos canónicos representada en formato JSON. Esta secuencia identifica los tipos de objetivo, las entidades, las ubicaciones y las relaciones extraídas de la instrucción. Posteriormente, un módulo determinista normaliza y valida la respuesta antes de convertirla en hechos que pueden ser procesados por CLIPS.

De esta manera, el modelo de lenguaje se utiliza específicamente para reducir la variabilidad lingüística presente en las instrucciones, mientras que la generación de las acciones necesarias para alcanzar los objetivos permanece bajo responsabilidad del componente simbólico.

Justificación de la selección de Qwen2.5-7B

La selección de Qwen2.5-7B como modelo base responde tanto a las características requeridas por la tarea como a las restricciones computacionales disponibles durante el desarrollo de esta investigación. La tarea planteada requiere interpretar instrucciones expresadas en lenguaje natural y transformarlas en una representación estructurada de objetivos canónicos, por lo que resulta conveniente utilizar un modelo con capacidad suficiente para seguir instrucciones, conservar las relaciones semánticas presentes en un orden y generar salidas con una estructura consistente.

Durante las etapas preliminares de desarrollo se realizaron pruebas con Qwen2.5-0.5B debido a sus menores requerimientos computacionales y a la posibilidad de efectuar rápidamente iteraciones de ajuste y evaluación. Estas pruebas tuvieron un carácter

exploratorio y permitieron validar inicialmente el flujo de entrenamiento y la representación utilizada para los objetivos.

Sin embargo, durante estas pruebas se observó que las respuestas del modelo presentaban una consistencia insuficiente para los requerimientos de la tarea, particularmente ante instrucciones con mayor complejidad o variabilidad lingüística. Debido a ello, se optó por utilizar una variante con mayor capacidad para los experimentos posteriores.

Estas observaciones iniciales se utilizaron como criterio de diseño para seleccionar la siguiente variante del modelo y no constituyen, por sí mismas, una comparación exhaustiva entre los diferentes tamaños de la familia Qwen2.5.

Se seleccionó finalmente Qwen2.5-7B como un compromiso entre capacidad del modelo y viabilidad computacional. Bajo la configuración de entrenamiento utilizada y los recursos computacionales disponibles durante esta investigación, la variante de siete mil millones de parámetros representó el mayor tamaño que resultó práctico para realizar de manera local los procesos de ajuste y experimentación. La plataforma utilizada estuvo compuesta por una GPU NVIDIA GeForce RTX 4070 Ti, un procesador Intel Core i7 de undécima generación y 32 GB de memoria RAM.

La utilización de modelos de mayor tamaño habría incrementado los requerimientos de memoria y procesamiento, además de reducir la flexibilidad para realizar múltiples iteraciones experimentales dentro de la infraestructura disponible. Por esta razón, se optó por mantener el modelo dentro de un tamaño que permitiera realizar tanto su adaptación como su evaluación localmente.

La elección de Qwen2.5-7B no pretende establecer que este modelo sea superior de manera general a otras familias de modelos de lenguaje. Existen diversas alternativas que podrían emplearse para una tarea de este tipo; sin embargo, la selección se realizó considerando el equilibrio entre capacidad para procesar instrucciones, disponibilidad del modelo, posibilidad de ejecutarlo y adaptarlo localmente, y los recursos computacionales disponibles. De esta manera, Qwen2.5-7B constituye una solución viable para evaluar la integración de un modelo de lenguaje especializado dentro de la arquitectura neuro-simbólica propuesta.

2.3.2. Modelos GPT

Los modelos GPT constituyen otra familia de modelos generativos basados en la arquitectura Transformer. En esta tesis, el modelo seleccionado se utiliza mediante la API de OpenAI, en lugar de desplegar sus parámetros dentro de la infraestructura local.

La API recibe la instrucción y el contexto proporcionado para la tarea y devuelve una respuesta generada por el modelo.

En el capítulo de implementación se especifican el modelo y la configuración utilizados durante los experimentos, debido a que una denominación general como “ChatGPT” no identifica de manera reproducible un modelo específico accesible mediante API (12).

En la arquitectura experimental GPT directo, el modelo recibe la orden, el catálogo de entidades, el repertorio de acciones, las restricciones del dominio y ejemplos de respuesta. A diferencia de Qwen implementado en la arquitectura neuro-simbólica, este modelo genera tanto la representación de los objetivos como la secuencia propuesta de acciones. Por ello, constituye un planificador generativo independiente y no un módulo interno de la cadena Qwen-CLIPS.

La respuesta del modelo se solicita mediante una salida estructurada definida a partir de JSON Schema. La documentación de la API denomina *Structured Outputs* al mecanismo mediante el cual la respuesta generada debe ajustarse al esquema proporcionado (12). Este mecanismo facilita la integración de la salida con los componentes posteriores del sistema, aunque el cumplimiento del esquema no garantiza por sí mismo la validez semántica del contenido generado.

Los distintos niveles de validación empleados para las salidas de los modelos se describen posteriormente en la subsección de salidas estructuradas y validación.

El uso de una API externa introduce variables que deben incluirse en la evaluación, como latencia de comunicación, disponibilidad del servicio, tokens de entrada y salida, coste monetario y posibles cambios entre versiones del modelo. En consecuencia, las pruebas deben fijar y documentar el modelo, los parámetros de generación y el prompt.

2.3.3. Interpretación semántica y generación de planes robóticos

Para esta tesis conviene distinguir dos funciones:

Interpretación semántica: Transformación de una orden en una representación intermedia de objetivos, entidades y relaciones.

Planificación generativa: Producción directa de una secuencia de acciones a partir de la orden, el contexto y las capacidades declaradas del robot.

En la arquitectura Qwen-CLIPS estas funciones se mantienen separadas: Qwen realiza la interpretación semántica de la orden y CLIPS expande los objetivos obtenidos

en una secuencia de acciones. En la arquitectura GPT directo, ambas funciones se concentran en un único modelo, que produce tanto la representación de los objetivos como el plan propuesto. Esta distinción permite comparar el efecto de mantener una etapa simbólica explícita frente a delegar conjuntamente la interpretación y la composición del plan a un modelo generativo.

La interpretación semántica puede evaluarse comparando la secuencia de objetivos producida por el modelo con una anotación de referencia. La planificación requiere criterios adicionales: pertenencia de las acciones al repertorio del robot, validez de parámetros, conservación de dependencias, cumplimiento de precondiciones y compatibilidad con el ejecutor. Evaluar ambas etapas de forma separada permite localizar el origen de los errores.

2.3.4. Limitaciones de los modelos de lenguaje en robótica

El uso de un modelo de lenguaje introduce riesgos específicos: generación de entidades o acciones inexistentes, incumplimiento del formato, variabilidad entre ejecuciones, sensibilidad al *prompt* y costes de inferencia. Asimismo, una instrucción ambigua puede dar lugar a una interpretación plausible pero diferente de la intención del usuario.

Estas limitaciones son especialmente relevantes cuando la salida se utiliza para controlar un sistema físico. Por ello, una respuesta lingüísticamente coherente no debe considerarse automáticamente un plan ejecutable. En esta investigación se separan la generación, la validación estructural y la validación semántica; los módulos de ejecución del robot conservan la responsabilidad sobre restricciones físicas y condiciones observadas durante la tarea.

2.3.5. Salidas estructuradas y validación

Una estrategia para integrar modelos de lenguaje con software convencional consiste en restringir la salida a una estructura legible por máquina. JSON permite representar objetivos, parámetros y acciones, mientras que JSON Schema permite comprobar tipos, campos obligatorios, enumeraciones y restricciones estructurales.

No obstante, una respuesta conforme al esquema puede seguir siendo semánticamente incorrecta. Por ejemplo, puede incluir una ubicación que no pertenece al mapa o aplicar una acción válida al tipo de entidad equivocado. En consecuencia, la validación debe separar al menos tres niveles:

1. Sintaxis JSON.
2. Conformidad con el esquema.
3. Validez semántica respecto a catálogos, acciones y precondiciones.

Esta separación es aplicable tanto a los objetivos producidos por Qwen como al plan generado directamente por GPT.

2.3.6. Adaptación eficiente de modelos: LoRA y QLoRA

El ajuste completo de un modelo de lenguaje requiere actualizar todos sus parámetros y mantener estados adicionales durante el entrenamiento. Los métodos de adaptación eficiente en parámetros permiten reducir este coste al entrenar únicamente un subconjunto reducido de parámetros, manteniendo congelada gran parte del modelo preentrenado (13, 14). LoRA (*Low-Rank Adaptation*) introduce matrices entrenables de bajo rango en determinadas capas del modelo, manteniendo congelados los pesos originales (14), mientras que QLoRA combina estos adaptadores con la cuantización del modelo base para reducir significativamente el uso de memoria durante el ajuste fino (15).

En esta investigación se utiliza QLoRA para especializar Qwen2.5-7B en la conversión de órdenes a objetivos canónicos. Los detalles del conjunto de datos, cuantización, hiperparámetros y plataforma pertenecen al capítulo de implementación.

2.4. Arquitecturas neuro-simbólicas

Las arquitecturas neuro-simbólicas buscan combinar las capacidades de aprendizaje y generalización de los modelos neuronales con mecanismos explícitos de representación de conocimiento y razonamiento simbólico. Mientras que los modelos neuronales son especialmente adecuados para procesar información no estructurada y aprender representaciones a partir de datos, los sistemas simbólicos permiten expresar conocimiento mediante estructuras discretas, reglas y relaciones que pueden ser utilizadas durante un proceso de inferencia (16, 17). El objetivo de este enfoque es aprovechar las fortalezas de ambos paradigmas en lugar de utilizar cada uno de manera aislada.

La integración entre ambos componentes puede realizarse de diferentes maneras. Un modelo neuronal puede, por ejemplo, transformar una entrada no estructurada en una representación simbólica que posteriormente es procesada por un sistema de razonamiento. En la dirección opuesta, el conocimiento simbólico puede utilizarse para

imponer restricciones, validar resultados o proporcionar información adicional durante el aprendizaje o la inferencia del modelo neuronal (17). Por tanto, una arquitectura neuro-simbólica no implica necesariamente que el aprendizaje neuronal y el razonamiento simbólico se realicen dentro de un único modelo.

Esta separación resulta particularmente útil cuando la entrada del sistema se encuentra expresada en lenguaje natural, pero la ejecución posterior requiere representaciones estructuradas y operaciones sujetas a reglas explícitas. En este tipo de arquitecturas, el modelo neuronal puede encargarse de interpretar la variabilidad lingüística y transformarla en una representación formal o canónica, mientras que un componente simbólico utiliza dicha representación para realizar razonamiento o planificación.

Un ejemplo de integración entre modelos de lenguaje y planificación simbólica es la arquitectura LLM+P, propuesta en *Empowering Large Language Models with Optimal Planning Proficiency* (18). En este enfoque, el modelo de lenguaje no se utiliza directamente como el mecanismo encargado de buscar la secuencia final de acciones. En su lugar, participa en la transformación de una descripción expresada en lenguaje natural hacia una representación formal que posteriormente puede ser procesada por un planificador clásico.

Para esta representación se utiliza PDDL (*Planning Domain Definition Language*), un lenguaje destinado a describir problemas de planificación automática. De manera general, PDDL permite representar los elementos que caracterizan al dominio de planificación, como las acciones disponibles, sus parámetros, precondiciones y efectos, así como los elementos específicos de un problema, entre ellos los objetos involucrados, el estado inicial y el objetivo que debe alcanzarse.

Una vez obtenida la representación formal del problema, la búsqueda de una secuencia de acciones que satisfaga el objetivo se delega a un planificador clásico. De esta manera, el modelo de lenguaje participa principalmente en la interpretación y formalización de la descripción proporcionada, mientras que el razonamiento necesario para obtener el plan se realiza mediante un mecanismo simbólico.

De manera simplificada, este flujo puede representarse como:

Lenguaje natural $\rightarrow$ Modelo de lenguaje $\rightarrow$ PDDL $\rightarrow$ Planificador clásico $\rightarrow$ Plan.

Esta separación permite asignar a cada componente una función distinta: el modelo de lenguaje se encarga principalmente de interpretar expresiones lingüísticas y convertirlas

en una representación estructurada, mientras que el planificador trabaja sobre dicha representación para determinar una secuencia de acciones que satisfaga las restricciones del problema (18). Este enfoque constituye un ejemplo de cómo un componente neuronal puede utilizarse como interfaz entre lenguaje natural y un mecanismo simbólico de planificación.

La arquitectura principal de esta tesis sigue una idea conceptualmente relacionada, aunque no utiliza PDDL ni reproduce directamente la arquitectura LLM+P. En este trabajo, el modelo Qwen2.5 (19), adaptado al dominio de tareas de servicio del robot, recibe una instrucción expresada en lenguaje natural y la transforma en una representación canónica de los objetivos solicitados. Esta representación constituye la interfaz entre el componente neuronal y el componente simbólico de la arquitectura.

El propósito de esta primera etapa es abstraer parte de la variabilidad presente en el lenguaje natural, como diferentes estructuras gramaticales, sinónimos o formas alternativas de expresar una misma intención. De este modo, distintas instrucciones que representan una intención equivalente pueden transformarse en objetivos con una estructura común antes de ser procesadas por el sistema de planificación.

En la segunda etapa, los objetivos obtenidos son enviados a CLIPS. A diferencia del modelo neuronal, CLIPS no interpreta directamente la instrucción original en lenguaje natural. Su función consiste en trabajar sobre los símbolos previamente obtenidos, incorporar los objetivos al estado de conocimiento y aplicar las reglas definidas para descomponerlos y expandirlos en las acciones necesarias para su ejecución. De esta manera, la interpretación lingüística y la expansión simbólica del plan permanecen como responsabilidades diferenciadas dentro de la arquitectura.

Por tanto, aunque LLM+P y la arquitectura propuesta en esta tesis utilizan mecanismos y representaciones diferentes, ambas ilustran una separación entre interpretación neuronal y razonamiento simbólico. En LLM+P, el modelo de lenguaje genera una representación PDDL que posteriormente es procesada por un planificador clásico; en esta tesis, Qwen genera objetivos canónicos que son posteriormente procesados mediante reglas de producción en CLIPS. La semejanza se encuentra, por tanto, en la separación funcional entre ambos paradigmas y no en el formalismo simbólico empleado.

Esta división permite que el modelo neuronal se concentre en el problema de correspondencia entre lenguaje natural y representaciones canónicas, mientras que el conocimiento sobre la estructura de las tareas, sus relaciones y su descomposición se mantiene explícitamente en la base de reglas de CLIPS. En consecuencia, una modificación en las

reglas de planificación no requiere necesariamente modificar el modelo de lenguaje y, de forma análoga, las mejoras realizadas sobre el componente de interpretación pueden introducirse sin sustituir el mecanismo simbólico de planificación.

En esta tesis, por tanto, la integración neuro-simbólica se describe como una composición secuencial: Qwen transforma el lenguaje natural en objetivos canónicos y CLIPS expande dichos objetivos mediante reglas de producción. Los componentes no actúan como dos planificadores concurrentes ni se realiza una fusión automática de planes generados independientemente. El denominado planificador GPT directo se implementa como una arquitectura alternativa con fines de comparación experimental y no como un componente adicional de la cadena neuro-simbólica Qwen-CLIPS.

2.5. Síntesis y conexión con la propuesta

Los antecedentes revisados muestran que los sistemas simbólicos permiten mantener una representación explícita del conocimiento utilizado durante la planificación, mientras que los modelos de lenguaje ofrecen mecanismos para procesar la variabilidad presente en las instrucciones expresadas en lenguaje natural. En una arquitectura híbrida, la combinación de ambos enfoques permite separar el problema de interpretación lingüística del proceso encargado de expandir los objetivos en acciones ejecutables.

Esta tesis estudia dicha separación mediante una comparación controlada entre una arquitectura secuencial Qwen-CLIPS y un planificador GPT directo. Adicionalmente, se utiliza una condición experimental de control en la que los objetivos de referencia son proporcionados directamente a CLIPS. Esta condición permite observar el comportamiento de la etapa simbólica cuando los objetivos de entrada corresponden directamente a la referencia, evitando introducir los errores provenientes de la etapa de interpretación del lenguaje natural. De esta manera, es posible distinguir los errores asociados con la interpretación lingüística de aquellos relacionados con la cobertura y expansión de las reglas de planificación. Esta condición no constituye una tercera arquitectura de planificación, sino un mecanismo auxiliar para aislar los errores correspondientes a la etapa simbólica.

El modelo Qwen2.5-7B ajustado mediante QLoRA recibe una instrucción y produce una secuencia de objetivos canónicos representada en JSON. Un módulo determinista valida y normaliza la respuesta, la convierte en hechos y la envía al motor CLIPS. Finalmente, las reglas de producción expanden cada objetivo en acciones compatibles

con el ejecutor del robot.

Como segunda arquitectura evaluada, se implementa un planificador basado en GPT que genera conjuntamente una representación estructurada de los objetivos interpretados y del plan propuesto. Su salida se restringe mediante JSON Schema y posteriormente se somete a validación determinista. Ambas arquitecturas se evalúan de forma independiente; por tanto, el trabajo no propone una selección concurrente ni una fusión automática de los planes generados.

La contribución experimental consiste en comparar ambos enfoques bajo el mismo conjunto de órdenes, catálogos y acciones, separando los errores de interpretación de los errores de planificación y cuantificando su desempeño mediante métricas de exactitud, ejecutabilidad, consistencia, latencia, uso de recursos y coste.

Capítulo 3

Formalización del problema y arquitectura conceptual

El problema abordado en esta tesis consiste en transformar una instrucción expresada en lenguaje natural en una secuencia de acciones que pueda ser procesada por el sistema de ejecución de un robot de servicio. Esta transformación requiere relacionar dos niveles de representación diferentes: por una parte, la intención expresada por el usuario y, por otra, las acciones concretas disponibles en el robot.

Como se revisó en el capítulo anterior, los modelos de lenguaje proporcionan mecanismos para procesar la variabilidad presente en las instrucciones lingüísticas, mientras que los sistemas simbólicos permiten representar conocimiento y relaciones mediante estructuras explícitas. A partir de estos antecedentes, en este capítulo se formaliza el problema particular considerado en la investigación y se establecen las representaciones utilizadas para describir órdenes, objetivos y planes.

El dominio experimental toma como referencia las tareas de General Purpose Service Robot (GPSR) de RoboCup@Home. Sin embargo, GPSR se utiliza como una fuente controlada de tareas, entidades y escenarios, y no como una restricción que obligue al usuario a reproducir literalmente las formulaciones lingüísticas utilizadas en la competencia.

Finalmente, se presentan las dos arquitecturas que serán evaluadas. La primera utiliza Qwen2.5-7B como intérprete semántico y CLIPS como componente encargado de expandir los objetivos obtenidos en acciones ejecutables. La segunda delega tanto la interpretación como la generación del plan a un modelo GPT. Ambas arquitecturas utilizan el mismo dominio conceptual, pero distribuyen de manera diferente la respon-

sabilidad de transformar una orden en un plan.

3.1. Contexto arquitectónico y alcance del sistema

3.1.1. Integración dentro de la arquitectura ViRBot

La investigación se desarrolla sobre el robot de servicio Justina y su arquitectura de software ViRBot (véase la Figura 3.1). Esta arquitectura integra diferentes capacidades necesarias para la operación del robot, entre ellas percepción, interacción humano-robot, navegación, manipulación, representación de conocimiento, planificación y ejecución.

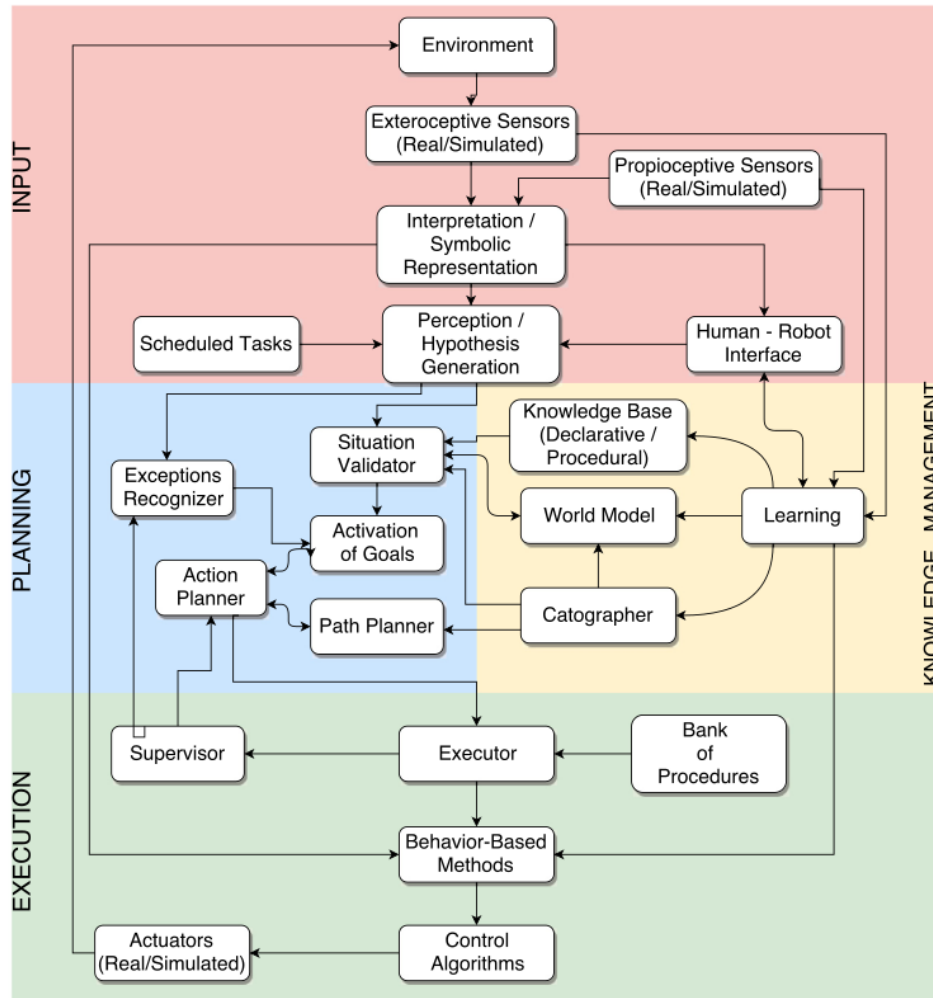


Figura 3.1: Arquitectura general ViRBot del robot Justina.

El sistema desarrollado en esta tesis se ubica entre la recepción de una instrucción expresada en lenguaje natural y los componentes encargados de ejecutar las acciones correspondientes. Por lo tanto, no pretende sustituir los módulos de navegación, percepción o manipulación del robot, sino proporcionar una representación ordenada de las acciones que dichos módulos deben realizar.

De manera simplificada, el flujo general puede expresarse como:

$$\boxed{\text{orden en lenguaje natural} \rightarrow \text{interpretación y planificación} \rightarrow \text{plan} \rightarrow \text{ejecución}} \quad (3.1)$$

La contribución estudiada se concentra en las etapas de interpretación y planificación anteriores a la ejecución física. Una instrucción puede contener varias subtarefas relacionadas que deben ser identificadas y transformadas en acciones compatibles con las capacidades del robot. Los módulos de percepción, navegación, manipulación e interacción encargados de realizar dichas acciones se consideran externos al problema de planificación estudiado. En consecuencia, la tesis evalúa principalmente la transformación de instrucciones en representaciones semánticas y planes de acción, mientras que la ejecución física se considera una etapa posterior que permite analizar, de forma separada, el comportamiento del plan sobre el robot.

3.2. GPSR como dominio de referencia

3.2.1. La prueba General Purpose Service Robot

General Purpose Service Robot (GPSR) es una prueba de RoboCup@Home orientada a evaluar la capacidad de un robot de servicio para comprender instrucciones expresadas en lenguaje natural y ejecutar tareas que pueden requerir la combinación de distintas habilidades. Entre ellas se encuentran la navegación, la búsqueda y reconocimiento de personas u objetos, la manipulación y la interacción humano-robot(20).

Una característica importante de GPSR es que una instrucción puede representar una tarea compuesta. Por ejemplo, una orden puede requerir que el robot se desplace hacia una habitación, localice un objeto, lo tome y posteriormente lo entregue a una persona. En consecuencia, el problema no consiste únicamente en reconocer palabras o entidades aisladas, sino en identificar las diferentes intenciones contenidas en la instrucción y conservar las relaciones existentes entre ellas.

En la prueba considerada en esta investigación se proporcionan tres comandos al robot. Al comenzar, el robot se desplaza hacia un *Instruction Point*, que funciona como el punto de interacción con un operador. Desde esta posición recibe una instrucción y, una vez que ha realizado la tarea solicitada, regresa al mismo punto para recibir el siguiente comando. La determinación de que una tarea ha sido completada de manera suficiente para continuar con la prueba corresponde a un árbitro.

La entrega de las órdenes se realiza de manera secuencial. Por tanto, el robot no recibe un nuevo comando mientras la ejecución de la tarea anterior continúe en curso. Una vez concluida la tarea y después de regresar al *Instruction Point*, puede solicitar la siguiente instrucción hasta completar los tres comandos contemplados por la prueba.

Cuando el robot no logra comprender adecuadamente la instrucción recibida mediante su sistema de entrada de audio, puede solicitar explícitamente que la orden sea repetida. En este caso, el operador expresará la misma tarea mediante una construcción lingüística más sencilla, pero manteniendo los objetivos contenidos en la instrucción original. La reformulación no constituye una modificación de la tarea ni permite que el operador indique directamente al robot cómo resolverla.

Este mecanismo es particularmente relevante para la presente investigación, pues permite distinguir entre la tarea que debe realizarse y la forma lingüística utilizada para comunicarla. Dos instrucciones con estructuras sintácticas diferentes pueden representar los mismos objetivos y, por lo tanto, deberían conducir a una interpretación semántica equivalente.

Durante la ejecución no se contempla que una persona intervenga para descomponer la orden, seleccionar las acciones del robot o corregir manualmente el plan generado. La interpretación de la instrucción y la determinación de las acciones necesarias corresponden al propio sistema robótico. De esta manera, GPSR proporciona un escenario apropiado para evaluar la transformación de instrucciones en lenguaje natural en representaciones semánticas y planes de acción.

3.2.2. Generación de comandos y conocimiento del dominio

Una característica relevante de GPSR es que los comandos utilizados durante la prueba no se seleccionan únicamente a partir de una lista fija de oraciones. El reglamento establece que las tareas se construyen mediante el *CommandGenerator* oficial de RoboCup@Home (20,21). Sobre esta herramienta se realizaron posteriormente adaptaciones específicas para construir el conjunto de datos empleado en el entrenamiento y

evaluación del modelo. Dichas modificaciones se describen en el capítulo de metodología.

El generador define diferentes familias de instrucciones relacionadas con personas y objetos. Entre ellas se encuentran tareas para desplazarse hacia una ubicación, encontrar personas u objetos, contar entidades, obtener información, seguir o guiar personas, tomar objetos y entregarlos o colocarlos en una ubicación determinada. Algunas de estas estructuras permiten incorporar continuaciones que producen comandos compuestos. De esta manera, una instrucción inicial puede extenderse con subtareas posteriores que dependen del resultado de las anteriores (21).

De forma simplificada, el proceso puede representarse como:

$$\boxed{\text{familia de tarea} + \text{estructura lingüística} + \text{entidades del dominio} \rightarrow \text{comando GPSR}} \quad (3.2)$$

Las estructuras del generador utilizan elementos variables para representar entidades y relaciones del dominio. Por ejemplo, una plantilla puede contener una habitación, una persona, un objeto o una ubicación que posteriormente son sustituidos por valores concretos. De forma similar, diferentes verbos pueden representar una intención relacionada; por ejemplo, una tarea de navegación puede expresarse mediante verbos como *go* o *navigate*, mientras que una búsqueda puede utilizar expresiones como *find*, *locate* o *look for* (21).

El conocimiento utilizado para realizar estas sustituciones se mantiene separado de las estructuras de los comandos. El generador distingue, entre otros elementos, nombres de personas, habitaciones, ubicaciones, superficies de colocación, objetos y categorías de objetos (21). La herramienta permite recibir este conocimiento desde un directorio externo y señala el repositorio *CompetitionTemplate* como referencia para su organización (21, 22).

En *CompetitionTemplate*, la información específica de una competencia se organiza en archivos relacionados con el mapa y las ubicaciones, los nombres de personas y los objetos disponibles (22). Esta separación resulta importante porque permite mantener las estructuras generales de las tareas mientras se modifican las entidades concretas presentes en una edición o escenario determinado.

Conceptualmente, el dominio de referencia combina las familias de tareas contempladas por el generador con el conocimiento específico utilizado para instanciarlas. Este último se representa mediante K y se formaliza en la Sección 3.3.3.

3.2.3. Dominio de referencia y alcance lingüístico

La utilización de GPSR como referencia no implica que el sistema desarrollado en esta tesis deba reconocer únicamente las frases producidas literalmente por el generador oficial. Esta distinción resulta especialmente importante debido a que una misma intención puede expresarse mediante diferentes construcciones lingüísticas.

De manera adicional, el reglamento de GPSR 2026 incorpora un mecanismo de variación sobre las órdenes generadas. Una vez producido un comando mediante el generador oficial, el reglamento establece que éste se proporciona a un modelo de lenguaje para obtener una formulación similar. Además, una instrucción puede ser reformulada hasta tres veces, utilizando expresiones progresivamente más simples cuando sea necesario (20).

Por ejemplo, el reglamento presenta una transformación equivalente para pasar de una instrucción como:

“Get me a coke from the kitchen.”

a una formulación más explícita, como:

“Go to the kitchen, find a coke, and bring it to me.”

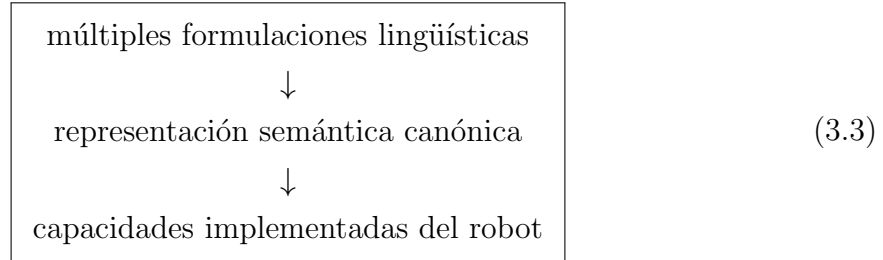
Aunque la estructura superficial de ambas expresiones es diferente, la tarea subyacente permanece esencialmente sin cambios. Esto permite distinguir dos niveles: forma lingüística e intención semántica.

Esta distinción es central para la presente investigación. GPSR proporciona un conjunto controlado de tareas y entidades, pero la entrada lingüística puede presentar diferentes formas de expresar una misma intención. El objetivo del intérprete no consiste, por tanto, en memorizar las plantillas del generador, sino en establecer una correspondencia entre distintas realizaciones lingüísticas y una representación semántica común.

Ambas formulaciones deben corresponder a la misma secuencia de objetivos. Esta propiedad se formaliza posteriormente mediante la noción de equivalencia semántica presentada en la Sección 3.3.2.

Por tanto, en esta tesis GPSR se utiliza como una fuente controlada para definir las tareas, entidades y relaciones que forman parte del dominio experimental, pero no como una gramática que limite las únicas formulaciones aceptables por el sistema.

Puede resumirse esta relación mediante:



En consecuencia, la entrada lingüística puede variar en su formulación, pero la interpretación permanece acotada por el dominio semántico y las capacidades operativas disponibles. Una instrucción puede ser perfectamente comprensible para una persona y, aun así, quedar fuera del alcance del sistema si requiere una intención que no puede representarse mediante los objetivos contemplados o una capacidad que el robot no posee.

El dominio experimental de esta investigación considera principalmente instrucciones en inglés, en correspondencia con las estructuras utilizadas por el generador y con el escenario de evaluación. Por ello, los resultados relacionados con variabilidad lingüística deben interpretarse como capacidad para procesar diferentes formulaciones de las tareas soportadas dentro de dicho idioma, y no como evidencia de independencia respecto al idioma o de comprensión irrestricta del lenguaje natural.

3.3. Formalización de órdenes, objetivos y planes

La transformación de una instrucción en lenguaje natural en un plan para un robot de servicio implica relacionar representaciones con diferentes niveles de abstracción. Por una parte, el usuario expresa una tarea mediante lenguaje natural; por otra, el sistema de ejecución requiere una secuencia de acciones compatibles con las capacidades disponibles del robot.

En esta investigación se introduce una representación semántica intermedia que permite separar ambos niveles. De manera general, el problema se representa mediante:

$$\boxed{x \rightarrow G \rightarrow P} \quad (3.4)$$

donde x representa la instrucción en lenguaje natural, G una secuencia ordenada de objetivos semánticos y P el plan compuesto por acciones ejecutables.

La primera transformación, $x \rightarrow G$, corresponde a la interpretación de la orden y

busca determinar qué solicita el usuario. La segunda, $G \rightarrow P$, corresponde a la planificación y determina qué acciones son necesarias para atender los objetivos identificados.

Esta separación constituye la base conceptual utilizada posteriormente para comparar las arquitecturas Qwen-CLIPS y GPT directo.

3.3.1. Orden en lenguaje natural

Se denota mediante x una instrucción expresada en lenguaje natural dentro del dominio considerado en esta investigación.

Una instrucción puede representar una tarea simple o una composición de varias subtarefas relacionadas. Por ejemplo:

“Go to the kitchen, find an apple and bring it to me.”

incluye navegación, búsqueda, manipulación y entrega de un objeto. Por tanto, una instrucción lingüística no corresponde necesariamente con una única acción del robot.

Incluso una expresión breve puede implicar varias operaciones. Considérese:

“Bring me an apple from the kitchen.”

Dentro del dominio utilizado en esta investigación, esta orden puede representarse mediante la siguiente secuencia semántica:

```
go(kitchen)
find(apple, kind=object)
take(apple)
deliver(apple, to=me)
```

La interpretación no consiste únicamente en identificar palabras clave. También debe reconocer las subtarefas explícitas o implícitas, las entidades involucradas y las relaciones necesarias entre ellas.

El resultado de este proceso es una representación semántica intermedia, denotada mediante G , cuya estructura se describe en la Sección 3.3.4.

3.3.2. Equivalencia semántica entre formulaciones

Como se describió en la Sección 3.2, una misma tarea puede expresarse mediante diferentes formulaciones lingüísticas. Esta propiedad resulta especialmente relevante en GPSR, donde una orden puede ser repetida o reformulada sin modificar los objetivos que debe cumplir el robot.

En esta investigación se consideran semánticamente equivalentes dos instrucciones cuando, dentro del dominio utilizado, representan la misma secuencia de objetivos.

Esta relación se denota mediante $x_1 \sim x_2$. Idealmente:

$$\boxed{x_1 \sim x_2 \quad \Rightarrow \quad G_1 = G_2} \quad (3.5)$$

donde G_1 y G_2 son las representaciones semánticas obtenidas para cada instrucción. Por ejemplo:

“Fetch a bottle from the living room and throw it in the trash.”

y:

“Go to the living room, grab a bottle, and toss it in the trash.”

presentan estructuras lingüísticas diferentes, pero pueden corresponder a la misma representación:

```
go(living room)
find(bottle, kind=object)
take(bottle)
place(bottle, in=trash)
```

Esta propiedad se utiliza en la presente investigación para representar de forma común órdenes lingüísticamente diferentes que conservan la misma intención o representación semántica canónica.

3.3.3. Entidades y conocimiento del dominio

La interpretación de una instrucción requiere información acerca de las entidades que pueden participar en las tareas consideradas.

En esta investigación, el conocimiento relevante del dominio se representa mediante:

$$K = \{K_P, K_O, K_C, K_L, K_R\}, \quad (3.6)$$

donde:

- K_P representa las personas o nombres conocidos;
- K_O representa los objetos;
- K_C representa las categorías de objetos;
- K_L representa las ubicaciones específicas;
- K_R representa las habitaciones.

Esta información permite determinar el tipo de las entidades utilizadas en una instrucción. Por ejemplo, si **Robin** pertenece al conjunto de personas y **apple** al conjunto de objetos, pueden utilizarse representaciones como:

```
find(Robin, kind=person)
find(apple, kind=object)
```

Ambos ejemplos utilizan el mismo tipo general de objetivo, pero operan sobre entidades semánticamente diferentes.

El conocimiento K cumple dos funciones principales. En primer lugar, permite verificar que las entidades nominales utilizadas pertenezcan al dominio. Una salida que introduzca arbitrariamente una persona, objeto o ubicación no debe considerarse correcta únicamente porque su estructura sea válida.

En segundo lugar, permite verificar la compatibilidad entre las entidades y los objetivos en los que participan.

Las referencias presentes en una instrucción pueden adoptar diferentes formas. En esta investigación se distinguen tres casos principales:

1. **Entidades nominales:** referencias explícitas a una entidad conocida, como **Robin**, **apple** o **kitchen**.
2. **Entidades genéricas:** referencias a una clase de entidad, como **person**, **drink** o **object**.
3. **Entidades descritas mediante propiedades:** referencias que requieren identificar una entidad a partir de alguna característica, como una persona saludando, una persona sentada o el objeto de mayor tamaño.

Por ejemplo:

```
find(person, kind=person, gesture='waving person')
```

no identifica inicialmente a una persona mediante un nombre, sino mediante una característica observable.

De manera similar:

```
find(drink, kind=object, property=largest)
```

representa la búsqueda de un objeto perteneciente a una categoría y sujeto a una propiedad adicional.

Estas distinciones permiten conservar la información necesaria para que las etapas posteriores puedan determinar cómo realizar la tarea.

3.3.4. Representación canónica de objetivos

El resultado de la interpretación se expresa mediante una secuencia ordenada de objetivos:

$$G = (g_1, g_2, \dots, g_n).$$

Cada objetivo g_i contiene un tipo de intención y los argumentos necesarios para describirla. Algunos ejemplos de objetivos individuales son:

```
go(kitchen)
find(Robin, kind=person)
deliver(apple, to=Robin)
```

Se denota mediante $\mathcal{G}$ el vocabulario de tipos de objetivos semánticos utilizados en esta investigación. La Tabla 3.1 resume estos objetivos y sus argumentos principales.

Los objetivos `find` y `count` requieren distinguir explícitamente si la entidad considerada corresponde a una persona o a un objeto. Para ello se utiliza el argumento `kind`. Por ejemplo:

```
find(Robin, kind=person)
find(apple, kind=object)
count(person, kind=person)
count(drinks, kind=object)
```

Tabla 3.1: Vocabulario de objetivos semánticos utilizado en la representación canónica.

Objetivo	Propósito	Argumentos principales
go	Desplazarse hacia una ubicación	ubicación
find	Localizar una persona u objeto	objetivo, kind , gesto, pose, vestimenta o propiedad
take	Tomar un objeto	objeto
deliver	Entregar un objeto	objeto, destinatario
place	Colocar un objeto en un destino	objeto, destino y relación espacial
guide	Guiar a una persona	persona, destino
follow	Seguir a una persona	persona, destino opcional
count	Contar personas u objetos	objetivo, kind y atributos opcionales
tell	Comunicar información	información o propiedad
save	Conservar información obtenida durante una interacción	nombre, pose o gesto
answer_question	Atender una pregunta	—
greet	Saludar a una persona	persona

Otros objetivos tienen un tipo de entidad implícito. Por ejemplo, **take**, **deliver** y **place** operan sobre objetos, mientras que **guide**, **follow** y **greet** operan sobre personas.

Los objetivos pueden incluir además calificadores relevantes presentes en la instrucción. Entre ellos se encuentran gestos, poses, vestimenta, características de objetos y relaciones espaciales. Por ejemplo:

```
find(person, kind=person, gesture='waving person')
find(person, kind=person, pose='sitting person')
find(person, kind=person, wearing='red shirt')
find(drink, kind=object, property=largest)
guide(Robin, to=kitchen)
place(apple, on=table)
```

La representación canónica busca conservar la información necesaria para describir la intención de la orden, eliminando al mismo tiempo las diferencias lingüísticas que no modifican su significado dentro del dominio.

La forma concreta utilizada para serializar y transmitir esta secuencia entre componentes pertenece a la implementación y se describe posteriormente.

Fidelidad de la representación semántica

Una secuencia formada únicamente por objetivos admitidos no constituye necesariamente una interpretación correcta.

Considérese nuevamente:

“Go to the kitchen and find Robin.”

La representación esperada es:

```
go(kitchen)
find(Robin, kind=person)
```

Una salida como:

```
find(Robin, kind=person)
```

utiliza un objetivo válido, pero omite la navegación solicitada.

Por otra parte:

```
go(kitchen)
find(Robin, kind=person)
greet(Robin)
```

incorpora una intención que no forma parte de la orden original.

En consecuencia, la fidelidad de la representación depende de que ésta conserve las subtareas, entidades, relaciones y dependencias expresadas o implicadas por la orden, sin introducir intenciones ajenas a ella.

Estos requisitos se formalizan posteriormente como criterios de validez en la Sección 3.5.2.

3.3.5. Objetivos semánticos y acciones ejecutables

Los objetivos canónicos describen **qué** debe conseguir el robot, pero no necesariamente especifican todas las operaciones concretas requeridas para hacerlo.

Se denota mediante $\mathcal{A}$ el repertorio de acciones disponibles para el sistema de ejecución. Entre ellas se encuentran acciones de navegación, interacción, búsqueda, percepción y manipulación, como `go_to`, `say`, `listen`, `find_person`, `find_object`, `approach`, `take`, `find_space`, `drop` y `give`.

El vocabulario de objetivos y el repertorio de acciones representan niveles diferentes:

$$\boxed{\mathcal{G} \neq \mathcal{A}} \quad (3.7)$$

Por ejemplo, el objetivo:

`deliver(apple, to=Robin)`

representa una intención de alto nivel. Para realizarla pueden ser necesarias acciones concretas destinadas a localizar al destinatario, aproximarse a él y entregar el objeto.

Conceptualmente:

`deliver` $\rightarrow$ `find_person` $\rightarrow$ `approach` $\rightarrow$ `give`.

De manera similar:

`place(apple, on=table)`

puede requerir identificar primero un espacio adecuado y posteriormente liberar el objeto mediante las acciones correspondientes.

El plan resultante se representa mediante una secuencia:

$$P = (a_1, a_2, \dots, a_m),$$

donde cada a_i corresponde a una acción perteneciente al repertorio $\mathcal{A}$.

Un objetivo puede producir una o varias acciones. Por ello, la planificación puede entenderse simplemente como la transformación de la secuencia de objetivos G en un plan P compatible con las capacidades disponibles:

$$G \rightarrow P.$$

Esta distinción permite evaluar por separado si una orden fue interpretada correctamente y si los objetivos obtenidos fueron transformados posteriormente en un plan adecuado.

3.3.6. Dependencias y contexto entre objetivos

La secuencia G no puede tratarse como una colección de objetivos independientes. Algunas subtarefas dependen de otras, reutilizan entidades identificadas previamente o establecen condiciones necesarias para objetivos posteriores.

Por esta razón, G se considera una **secuencia ordenada**.

Dependencias de precedencia

Algunos objetivos requieren que una subtarea anterior haya sido realizada.

Por ejemplo:

```
take(apple)
deliver(apple, to=Robin)
```

requiere adquirir el objeto antes de intentar entregarlo. Esta relación se representa mediante:

$$\text{take}(o) \prec \text{deliver}(o, p), \quad (3.8)$$

donde $\prec$ indica que el objetivo situado a la izquierda debe preceder al situado a la derecha.

Una secuencia que invirtiera esta relación podría contener objetivos individualmente válidos, pero no representaría una tarea coherente.

Dependencias referenciales

Algunas instrucciones introducen entidades que posteriormente deben ser reutilizadas.

Considérese:

```
find(person, kind=person, gesture='waving person')
guide(person, to=kitchen)
```

La persona utilizada en el segundo objetivo no representa una persona arbitraria. Se refiere a la entidad identificada por la búsqueda anterior.

Este tipo de relación permite representar instrucciones que contienen pronombres, referencias genéricas o descripciones cuyo referente se obtiene durante el desarrollo de la tarea.

Dependencias espaciales y de estado

Algunos objetivos establecen un contexto que debe ser considerado por las subtarefas posteriores.

Por ejemplo:

```
go(kitchen)
find(apple, kind=object)
```

indica que la búsqueda del objeto debe realizarse después de desplazarse hacia la cocina. El objetivo de navegación establece así el contexto espacial esperado para la búsqueda posterior.

De manera similar:

```
take(apple)
place(apple, on=table)
```

establece una condición intermedia: después de tomar la manzana, el plan puede considerar que el robot dispone del objeto para realizar su colocación.

Estas condiciones representan el **estado esperado durante la planificación**. No constituyen una confirmación de que las acciones hayan sido ejecutadas correctamente en el mundo físico. Esta diferencia se retoma posteriormente al distinguir entre planificación y ejecución.

Orden lingüístico y orden semántico

Finalmente, debe distinguirse el orden en el que las palabras aparecen dentro de una oración del orden lógico requerido para realizar la tarea.

Por ejemplo:

“Bring me the apple from the kitchen.”

expresa directamente una intención de entrega, pero para realizarla puede ser necesario construir la siguiente secuencia:

```
go(kitchen)
find(apple, kind=object)
take(apple)
deliver(apple, to=me)
```

El orden lógico puede resumirse como:

$$\text{go} \prec \text{find} \prec \text{take} \prec \text{deliver}.$$

La interpretación no consiste, por tanto, en reproducir el orden superficial de las palabras, sino en recuperar las relaciones necesarias entre las subtareas.

Esta propiedad resulta especialmente relevante en instrucciones compactas, en las que algunas operaciones requeridas para cumplir la intención pueden no aparecer expresadas como verbos independientes.

En conjunto, la representación G debe conservar tanto la intención de la orden como las entidades, relaciones y dependencias necesarias para transformarla posteriormente en un plan coherente.

3.4. Arquitecturas conceptuales evaluadas

A partir de la formalización presentada en la Sección 3.3, las dos arquitecturas consideradas en esta investigación resuelven el mismo problema, pero distribuyen de manera diferente las responsabilidades de interpretación y planificación.

En la arquitectura Qwen–CLIPS, ambas funciones se encuentran separadas: Qwen realiza la interpretación semántica y CLIPS transforma posteriormente los objetivos obtenidos en acciones ejecutables.

En la arquitectura GPT directo, un mismo modelo realiza tanto la interpretación de la orden como la generación del plan.

Esta diferencia constituye el eje principal de la comparación. Por tanto, el objetivo no consiste únicamente en comparar dos modelos de lenguaje, sino en contrastar dos formas distintas de organizar el proceso que relaciona lenguaje natural, objetivos semánticos y acciones ejecutables.

3.4.1. Base común de las arquitecturas evaluadas

Para que la comparación sea significativa, ambas arquitecturas operan sobre una misma definición del problema y comparten los elementos conceptuales establecidos en la sección anterior.

En particular, ambas utilizan:

- el mismo conocimiento del dominio K ;

- el mismo vocabulario de objetivos semánticos $\mathcal{G}$;
- el mismo repertorio de acciones ejecutables $\mathcal{A}$;
- las mismas familias de tareas consideradas para la evaluación;
- una representación común de referencia para determinar qué objetivos corresponden a cada instrucción.

En consecuencia, ambas arquitecturas deben interpretar órdenes pertenecientes al mismo dominio y producir planes compatibles con las mismas capacidades del robot.

Sin embargo, utilizar el mismo dominio no significa que el conocimiento necesario para resolver la tarea se encuentre distribuido de la misma manera.

En Qwen–CLIPS, Qwen debe determinar únicamente qué solicita el usuario. El conocimiento necesario para convertir posteriormente esos objetivos en acciones se encuentra representado mediante reglas dentro de CLIPS.

En cambio, GPT directo debe disponer durante una misma inferencia tanto del conocimiento necesario para interpretar la orden como de las restricciones necesarias para generar el plan.

Por tanto, la comparación se realiza sobre un mismo problema y un mismo resultado esperado, aunque cada arquitectura distribuya de manera diferente el conocimiento utilizado para alcanzarlo.

3.4.2. Arquitectura Qwen–CLIPS

La primera arquitectura adopta una separación explícita entre interpretación semántica y planificación.

Su funcionamiento general puede resumirse mediante:

$$\boxed{x \xrightarrow{\text{Qwen}} G \xrightarrow{\text{CLIPS}} P} \quad (3.9)$$

Qwen actúa como intérprete semántico. Su responsabilidad consiste en recibir la instrucción en lenguaje natural y producir una secuencia ordenada de objetivos pertenecientes al vocabulario definido en $\mathcal{G}$. No determina directamente las acciones ejecutables; su responsabilidad termina en la construcción de la representación semántica definida en la Sección 3.3.4.

Antes de ser procesados por CLIPS, los objetivos obtenidos son sometidos a comprobaciones deterministas destinadas a verificar que utilizan la estructura, los tipos de objetivos y las entidades admitidas por el dominio. Los detalles de estas comprobaciones

y de la representación utilizada para intercambiar la información entre componentes se presentan posteriormente en el capítulo de implementación.

La Figura 3.2 resume el flujo conceptual de esta arquitectura.

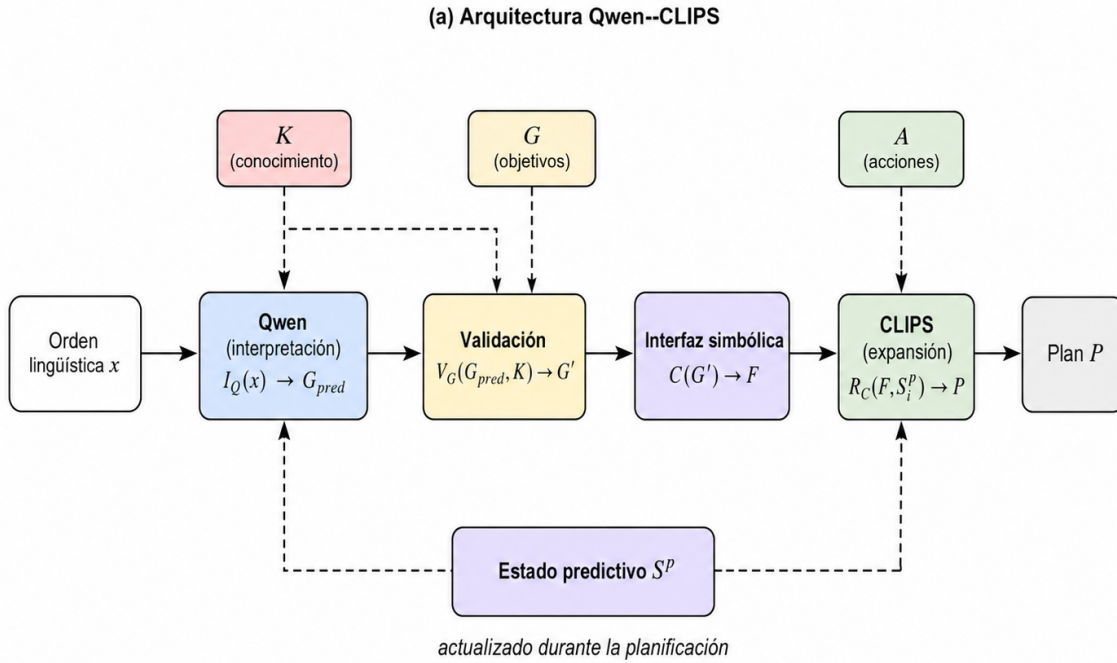


Figura 3.2: Arquitectura conceptual Qwen–CLIPS.

Una vez aceptada la representación semántica, CLIPS se encarga de transformar los objetivos en acciones pertenecientes al repertorio $\mathcal{A}$.

En esta arquitectura, CLIPS funciona como un expansor de objetivos basado en reglas de producción. Su función no consiste en interpretar nuevamente la orden original ni en realizar una búsqueda libre entre planes posibles. En cambio, cada objetivo activa conocimiento explícito que determina las acciones necesarias para atenderlo, siguiendo la distinción entre objetivos semánticos y acciones ejecutables presentada en la Sección 3.3.5.

La expansión conserva el orden y las relaciones presentes en G , de manera que las dependencias descritas en la Sección 3.3.6 puedan ser consideradas durante la generación

del plan.

Estado esperado durante la planificación

Durante la expansión de los objetivos, CLIPS utiliza el estado esperado descrito en la Sección 3.3.6 para conservar la coherencia entre subtareas consecutivas. De esta manera, las decisiones asociadas con un objetivo pueden considerar los resultados esperados de objetivos procesados previamente.

En esta arquitectura, dicha información forma parte del proceso simbólico de expansión. Su función se limita a mantener consistencia durante la construcción del plan.

Separación de errores

La separación entre Qwen y CLIPS permite identificar dos fuentes principales de error. La primera corresponde a la interpretación semántica, cuando los objetivos producidos no representan correctamente la orden. La segunda corresponde a la planificación simbólica, cuando una representación correcta no puede transformarse adecuadamente en acciones ejecutables.

Además, una interpretación incorrecta puede producir posteriormente un plan internamente coherente con objetivos equivocados. Por ello, la evaluación de esta arquitectura debe considerar de forma independiente la corrección de G y la corrección de P .

Esta separación se utiliza posteriormente para definir la condición experimental de control presentada en la Sección 3.5.5.

3.4.3. Arquitectura de planificación directa con GPT

La segunda arquitectura concentra en un mismo modelo las funciones de interpretación semántica y generación del plan.

Su comportamiento principal puede resumirse mediante:

$$\boxed{x \xrightarrow{\text{GPT}} (G, P)} \quad (3.10)$$

Para una instrucción ejecutable, GPT produce en una misma respuesta tanto la secuencia de objetivos semánticos G como el plan de acciones P .

La conservación explícita de ambos elementos resulta importante. Aunque hayan sido generados por el mismo modelo, G representa la interpretación realizada sobre la

orden, mientras que P representa la secuencia de acciones propuesta para atenderla.

De esta manera, ambos resultados pueden ser analizados posteriormente de forma independiente.

La Figura 3.3 presenta el flujo conceptual de esta arquitectura.

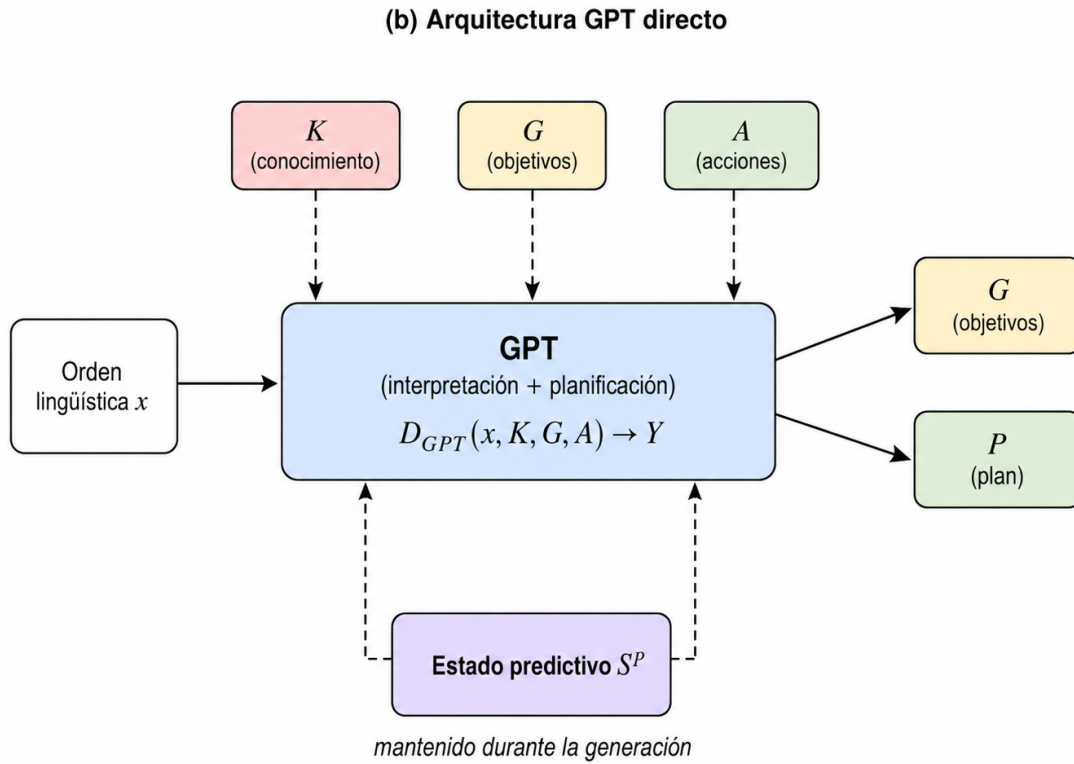


Figura 3.3: Arquitectura conceptual del planificador GPT directo.

Una diferencia fundamental respecto a Qwen–CLIPS es que los objetivos producidos por GPT no se utilizan como entrada para CLIPS. Tanto G como P son generados durante una misma inferencia y se conservan como resultados diferenciados para su posterior evaluación.

Tipos de respuesta

La arquitectura no obliga al modelo a producir un plan para cualquier instrucción recibida. Dependiendo de la información disponible y de la compatibilidad de la orden con el dominio, se contemplan tres tipos de respuesta:

- **executable:** la instrucción contiene información suficiente y puede representarse mediante objetivos y acciones soportadas. En este caso se generan G y P .
- **clarification:** existe información faltante o una ambigüedad que puede resolverse mediante una pregunta breve al usuario. En este caso no se genera un plan hasta resolver dicha ambigüedad.
- **rejected:** la instrucción se encuentra fuera del dominio, requiere una capacidad no disponible o contradice las restricciones establecidas. En este caso tampoco se genera un plan.

Esta clasificación permite evitar que el modelo genere arbitrariamente una secuencia de acciones cuando no dispone de información suficiente para interpretar la orden.

Planificación directa bajo restricciones

El término *planificación directa* indica que GPT produce las acciones del plan sin delegar posteriormente la expansión de los objetivos a CLIPS.

Esto no significa que el modelo pueda generar cualquier comportamiento.

La generación se encuentra restringida mediante información sobre:

- las entidades admitidas en el dominio;
- los tipos de objetivos pertenecientes a $\mathcal{G}$;
- las acciones disponibles en $\mathcal{A}$;
- las relaciones y dependencias necesarias entre subtareas;
- las restricciones que determinan qué combinaciones pueden constituir un plan válido.

Así, una intención semántica como **deliver** no debe utilizarse directamente como una acción ejecutable si no pertenece al repertorio $\mathcal{A}$. El modelo debe generar las acciones concretas necesarias para realizar dicha intención.

El conocimiento que en Qwen-CLIPS se encuentra expresado mediante reglas de producción debe encontrarse disponible para GPT mediante las instrucciones y restricciones proporcionadas durante la generación.

Esta diferencia no modifica el dominio de la comparación, sino la forma en que cada arquitectura representa y utiliza el conocimiento necesario para construir el plan.

Estado esperado durante la generación

GPT también debe mantener coherencia con el estado esperado descrito en la Sección 3.3.6 al generar las acciones del plan.

La diferencia respecto a Qwen–CLIPS se encuentra en el mecanismo utilizado. En la arquitectura simbólica, las relaciones necesarias para mantener esta coherencia se representan mediante conocimiento y reglas explícitas. En GPT directo, el modelo debe conservarlas durante la generación condicionada por el contexto y las restricciones proporcionadas.

Separación posterior de interpretación y planificación

Aunque GPT realice interpretación y planificación durante una misma inferencia, la conservación de G y P permite analizar posteriormente ambas funciones por separado.

Si los objetivos producidos no corresponden con los objetivos de referencia, el error puede atribuirse principalmente a la interpretación de la orden.

En cambio, si G representa correctamente la instrucción pero P omite acciones necesarias, utiliza acciones no soportadas o viola dependencias de la tarea, puede identificarse un error de planificación.

A diferencia de Qwen–CLIPS, esta separación no corresponde a dos componentes independientes del sistema. Se obtiene mediante el análisis de los dos artefactos generados por el mismo modelo.

3.4.4. Comparación conceptual de las arquitecturas

La diferencia esencial entre las arquitecturas puede resumirse mediante:

$$\begin{array}{l} \text{Qwen–CLIPS: } x \rightarrow G \rightarrow P \\ \text{GPT directo: } x \rightarrow (G, P) \end{array} \quad (3.11)$$

En Qwen–CLIPS, la interpretación y la planificación corresponden a etapas explícitamente separadas: Qwen produce la representación semántica y CLIPS transforma posteriormente dicha representación en acciones. En GPT directo, ambas funciones se realizan durante una misma inferencia, aunque G y P se conservan como resultados diferenciados para permitir su análisis posterior.

La Tabla 3.2 resume las principales características de ambas estrategias.

El carácter basado en reglas de CLIPS no implica que cualquier plan producido sea necesariamente correcto ni garantiza el éxito de su ejecución física. De manera similar, restringir la estructura de la respuesta de GPT no garantiza que la interpretación o el plan generado sean semánticamente correctos.

Tabla 3.2: Comparación conceptual entre las arquitecturas evaluadas.

Característica	Qwen-CLIPS	GPT directo
Interpretación semántica	Qwen	GPT
Generación del plan	CLIPS	GPT
Representación explícita de G	Sí	Sí
Generación de P	Reglas de producción	Generación condicionada del modelo
Conocimiento para expandir los objetivos	Representado explícitamente mediante reglas	Proporcionado mediante contexto y restricciones
Separación entre interpretación y planificación	Explícita entre componentes	Integrada en una misma inferencia
Estado durante la planificación	Predictivo y simbólico	Predictivo durante la generación
Dependencia de un servicio externo de terceros	No	Sí
Identificación de errores	Separación directa entre etapas	Separación posterior mediante G y P
Variabilidad del resultado	Depende de los objetivos, reglas y condiciones iniciales	Puede presentar variación entre inferencias

Por ello, en ambas arquitecturas resulta necesario evaluar por separado la representación semántica G y el plan P . Esta separación permite analizar no sólo qué arquitectura produce una mayor proporción de resultados correctos, sino también en qué etapa se originan los errores, qué grado de consistencia presenta cada estrategia y qué recursos requiere para transformar una instrucción en un plan compatible con las capacidades del robot.

3.5. Criterios conceptuales de validez

La generación de una salida correctamente estructurada no implica, por sí misma, que la interpretación de una orden o el plan correspondiente sean correctos.

Por esta razón, en esta investigación se distinguen diferentes niveles de validez. El propósito de esta separación es identificar si un error se origina durante la interpretación de la instrucción, durante la generación del plan o posteriormente durante su ejecución física.

De manera general, el proceso puede representarse como:

$$\boxed{x \xrightarrow{\text{interpretación}} G \xrightarrow{\text{planificación}} P \xrightarrow{\text{ejecución}} E} \quad (3.12)$$

donde x representa la instrucción en lenguaje natural, G la secuencia de objetivos semánticos, P el plan generado y E el resultado observado durante la ejecución.

Cada una de estas etapas debe evaluarse de manera independiente.

3.5.1. Niveles de validez

Para analizar las salidas producidas por las arquitecturas evaluadas se consideran los siguientes niveles:

1. **Validez sintáctica:** la salida puede ser procesada por el sistema y respeta la sintaxis esperada.
2. **Validez estructural:** los objetivos, acciones, argumentos y tipos utilizados pertenecen a las estructuras admitidas por el sistema.
3. **Validez semántica:** la representación G conserva la intención expresada en la instrucción original.
4. **Validez del plan:** la secuencia P constituye una realización coherente de los objetivos representados en G .
5. **Éxito de ejecución:** el robot consigue realizar físicamente las acciones planificadas y alcanzar el resultado esperado.

Estos niveles no deben considerarse equivalentes. Una salida puede ser sintáctica y estructuralmente válida y, aun así, representar incorrectamente la intención del usuario. De manera similar, un plan puede ser conceptualmente correcto pero fallar durante la ejecución debido a condiciones externas al proceso de planificación.

Esta separación permite evitar que todos los fallos observados sean agrupados bajo una única categoría de *plan incorrecto*.

3.5.2. Validez de la interpretación semántica

La interpretación se considera válida cuando la secuencia de objetivos G representa fielmente la tarea expresada en la instrucción x dentro del dominio considerado.

Los requisitos de fidelidad fueron introducidos en la Sección 3.3.4. Para la evaluación conceptual se consideran principalmente los siguientes criterios:

1. **Cobertura de las subtareas:** deben conservarse todos los objetivos necesarios para representar la tarea, incluyendo aquellas operaciones implícitas que resulten

necesarias dentro del dominio.

2. **Ausencia de intenciones no solicitadas:** la representación no debe introducir objetivos adicionales que modifiquen la intención original de la orden.
3. **Exactitud de entidades y tipos:** las personas, objetos, categorías y ubicaciones utilizadas deben corresponder con las referencias presentes en la instrucción y ser compatibles con el conocimiento K .
4. **Conservación de relaciones y dependencias:** deben mantenerse las relaciones de precedencia, referencia y contexto necesarias entre los objetivos.

Cuando se dispone de una representación de referencia G_{ref} , la interpretación producida por el sistema puede compararse con ella. El caso ideal corresponde a:

$$G_{\text{pred}} = G_{\text{ref}}.$$

Esta condición representa una correspondencia estricta entre la interpretación obtenida y la esperada. Durante la evaluación puede complementarse con criterios más específicos para identificar omisiones, adiciones, errores de entidad, relaciones incorrectas u otros tipos de discrepancia.

Consistencia ante reformulaciones

La variabilidad lingüística introduce un criterio adicional de validez. Como se definió en la Sección 3.3.2, formulaciones semánticamente equivalentes deben conducir idealmente a la misma representación canónica.

Esta propiedad resulta especialmente relevante en GPSR, donde una misma instrucción puede ser reformulada sin modificar sus objetivos.

Por tanto, la evaluación debe considerar no sólo la interpretación de una formulación aislada, sino también la consistencia del sistema ante diferentes expresiones de una misma intención.

3.5.3. Validez y consistencia del plan

Una vez obtenida la representación semántica G , debe determinarse si el plan P constituye una realización coherente de dichos objetivos.

Un plan se considera conceptualmente válido cuando cumple los siguientes criterios:

1. **Acciones soportadas:** todas las acciones utilizadas pertenecen al repertorio $\mathcal{A}$ disponible para el sistema de ejecución.

2. **Parámetros válidos:** cada acción utiliza argumentos compatibles con su función y con las entidades del dominio.
3. **Respeto de dependencias:** la secuencia conserva el orden y las relaciones necesarias para realizar la tarea.
4. **Cobertura de los objetivos:** las acciones generadas permiten atender los objetivos contenidos en G sin omitir partes necesarias de la tarea.
5. **Coherencia con el estado esperado:** las acciones posteriores son compatibles con los resultados esperados de las acciones previamente incluidas en el plan.

Por lo tanto, una secuencia puede estar formada únicamente por acciones válidas y, aun así, constituir un plan incorrecto si presenta un orden inadecuado, utiliza parámetros incompatibles o no cubre completamente los objetivos solicitados.

3.5.4. Validez de la respuesta del sistema

Además de evaluar la corrección de G y P , en la arquitectura GPT directo debe considerarse si el modelo selecciona adecuadamente el tipo de respuesta descrito en la Sección 3.4.3.

Una respuesta **executable** es apropiada cuando la instrucción contiene información suficiente y puede resolverse mediante las capacidades disponibles. Una respuesta **clarification** es apropiada cuando existe una ambigüedad que puede resolverse solicitando información adicional, mientras que **rejected** corresponde a instrucciones fuera del dominio o de las capacidades admitidas.

Por ejemplo:

“Bring it to her.”

puede requerir una aclaración si no existe suficiente contexto para determinar qué representan *it* y *her*.

Por tanto, generar un plan no constituye siempre la respuesta correcta. Ante una orden ambigua o no soportada, solicitar una aclaración o rechazar la solicitud puede representar el comportamiento esperado del sistema.

3.5.5. Condición de control para la etapa simbólica

La separación explícita entre interpretación y planificación de la arquitectura Qwen-CLIPS permite introducir una condición experimental de control destinada a aislar el comportamiento del componente simbólico.

En esta condición, los objetivos de referencia se proporcionan directamente a CLIPS:

$$\boxed{G_{\text{ref}} \xrightarrow{\text{CLIPS}} P_{\text{CLIPS}}} \quad (3.13)$$

Esta condición no constituye una tercera arquitectura de planificación. Su propósito es evaluar únicamente la transformación $G \rightarrow P$, eliminando temporalmente los posibles errores producidos durante $x \rightarrow G$.

Si CLIPS genera un plan correcto cuando recibe G_{ref} , pero el flujo completo Qwen-CLIPS falla para la misma instrucción, el error puede encontrarse en la interpretación producida por Qwen.

En cambio, si el plan generado a partir de G_{ref} también es incorrecto, el problema corresponde a la expansión simbólica, la cobertura de las reglas o la generación del plan.

Esta condición permite distinguir experimentalmente entre dos tipos principales de error:

- **error de interpretación:** la secuencia G no representa correctamente la instrucción;
- **error de planificación:** la representación G es correcta, pero el plan P no constituye una expansión adecuada.

Esta separación será utilizada posteriormente para clasificar los resultados obtenidos durante la evaluación.

3.5.6. Planificación y ejecución física

El último nivel de análisis corresponde a la diferencia entre generar un plan correcto y completar satisfactoriamente su ejecución sobre el robot.

La relación fundamental puede expresarse de manera sencilla como:

$$\boxed{\text{plan correcto} \not\Rightarrow \text{ejecución exitosa}} \quad (3.14)$$

Un plan puede representar correctamente la intención de la instrucción y contener una secuencia coherente de acciones, pero su ejecución puede fallar por factores externos al proceso de planificación.

Entre ellos se encuentran, por ejemplo:

- fallos durante la percepción de personas u objetos;
- obstáculos o cambios en el entorno durante la navegación;
- dificultades durante la manipulación de objetos;

- cambios en la posición de personas u objetos;
- problemas durante la interacción humano-robot.

Por ejemplo, el plan puede indicar correctamente que el robot debe tomar una manzana, pero la ejecución puede fallar si el objeto no es detectado o si el manipulador no consigue sujetarlo.

De manera similar, una acción de navegación puede estar correctamente planificada y, aun así, no completarse debido a obstáculos o modificaciones del entorno.

Por esta razón, el estado esperado utilizado durante la planificación no debe confundirse con el estado real observado durante la ejecución.

La evaluación de las arquitecturas debe distinguir, por tanto, entre:

1. la correcta interpretación de la instrucción;
2. la generación de un plan coherente con dicha interpretación;
3. el resultado obtenido al intentar ejecutar físicamente ese plan.

Esta separación permite analizar los resultados sin atribuir automáticamente a la interpretación o a la planificación fallos originados en otros componentes del robot.

Los criterios establecidos en esta sección definen conceptualmente qué se considera una interpretación correcta, un plan válido y una ejecución exitosa. Las métricas concretas, procedimientos experimentales y criterios utilizados para medir estas propiedades se presentan posteriormente en el capítulo de validación y experimentación.

Capítulo 4

Metodología y diseño de las arquitecturas evaluadas

En esta investigación se desarrolló una arquitectura para transformar instrucciones en lenguaje natural en planes de acciones para el robot Justina, mediante la especialización de Qwen2.5-7B como intérprete semántico y la expansión de objetivos mediante reglas CLIPS. Para estudiar esta estrategia se diseñó una arquitectura de comparación en la que un modelo GPT genera los objetivos y el plan durante una misma inferencia.

El desarrollo requirió adaptar el generador de comandos GPSR, construir un conjunto de datos con etiquetas semánticas, establecer una interfaz entre la interpretación y la planificación, y expresar el conocimiento operativo en las dos arquitecturas. Estas decisiones se apoyan en las representaciones y los criterios definidos en el Capítulo 3.

Este capítulo explica cómo se construyó el dominio de trabajo, qué modificaciones se realizaron para obtener el conjunto de datos y cómo se diseñó la comparación entre las arquitecturas. Los detalles de código, entrenamiento y despliegue se presentan en el capítulo 5. El protocolo de evaluación, las métricas y las condiciones concretas de los experimentos se desarrollan en el capítulo 6.

4.1. Diseño metodológico de la investigación

4.1.1. Problema experimental

La investigación estudia la fidelidad y la consistencia con las que dos arquitecturas transforman instrucciones de un dominio común en planes compatibles con las capa-

ciudades de Justina. La comparación considera tanto la recuperación de las subtareas solicitadas como su realización mediante acciones, con especial atención a las entidades y dependencias que intervienen en órdenes compuestas.

La unidad básica de análisis es una instrucción x , asociada con una secuencia de objetivos de referencia G_{ref} . Se conserva la notación G_{pred} del capítulo anterior para representar una secuencia de objetivos predicha. Cuando sea necesario distinguir su procedencia, se utilizarán $G_{\text{pred}}^{\text{Qwen}}$ y $G_{\text{pred}}^{\text{GPT}}$. Los superíndices identifican el modelo que produce la representación.

En Qwen-CLIPS, la predicción $G_{\text{pred}}^{\text{Qwen}}$ se somete a validación y conversión antes de su expansión mediante CLIPS, que produce el plan $P^{\text{Qwen-CLIPS}}$. En la arquitectura directa, GPT produce $G_{\text{pred}}^{\text{GPT}}$ y P^{GPT} durante una misma inferencia. Estos productos permiten examinar la interpretación frente a una referencia común y analizar después la correspondencia entre los objetivos y las acciones.

La evaluación se orienta a determinar en qué tareas aparecen diferencias de comportamiento y qué tipo de errores explica esas diferencias. Cuando se utilizan varias formulaciones de una misma intención, se considera también la estabilidad de la representación obtenida ante los cambios lingüísticos.

4.1.2. Variables y condiciones de comparación

El factor principal de comparación es la arquitectura completa. En Qwen-CLIPS, la interpretación se obtiene mediante un modelo especializado y la expansión se realiza con reglas explícitas. En GPT directo, ambas funciones se asignan a un modelo condicionado por una especificación del dominio y del comportamiento operativo esperado.

Las variables de respuesta comprenden la corrección de los objetivos, la validez y consistencia del plan y el rendimiento computacional. La familia de tarea, la longitud de la secuencia de objetivos, los tipos de entidades y las dependencias referenciales se utilizan para organizar el análisis. Las variantes que expresan una misma intención se mantienen identificadas como tales para interpretar adecuadamente los resultados por caso y por grupo de reformulaciones.

La comparación se delimita a instrucciones en inglés, un vocabulario semántico común y un repertorio compartido de acciones. El diseño exige igualdad de catálogos, configuración de manipulación y convenciones de inicio y finalización. Estas condiciones permiten plantear la misma tarea a ambos sistemas, aunque cada uno utilice mecanismos distintos para representar y aplicar el conocimiento.

Dado que entre las arquitecturas cambian también el modelo de lenguaje y su adaptación, las diferencias observadas no pueden atribuirse exclusivamente a la presencia de reglas. Las conclusiones se referirán al comportamiento de las configuraciones evaluadas dentro del dominio establecido.

4.1.3. Estrategia general de evaluación

La evaluación se organiza a partir de los niveles de validez definidos en la Sección 3.5. Se examina primero la interpretación de la orden y después la generación del plan. Las pruebas físicas con Justina se consideran una etapa adicional, de manera que los fallos de ejecución puedan analizarse por separado de los errores anteriores.

Para estudiar la etapa simbólica se incorpora la condición de control de la Sección 3.5.5, en la que se proporcionan los objetivos de referencia a CLIPS. Esta condición permite contrastar el resultado de la expansión con el obtenido a partir de la interpretación de Qwen. La inspección de los objetivos y de su conversión a hechos ayuda a distinguir errores de interpretación, integración y expansión.

Se separan asimismo las comprobaciones realizadas durante el desarrollo de la evaluación comparativa. Las primeras permiten verificar el funcionamiento del sistema y corregir problemas de implementación. La segunda requiere instrucciones y referencias cuyo uso no haya intervenido en el ajuste de los componentes que se comparan.

4.2. Construcción del dominio experimental

4.2.1. Fuentes de conocimiento GPSR

Se tomó como punto de partida el CommandGenerator oficial de RoboCup@Home y la organización del conocimiento de CompetitionTemplate, descritos en la Sección 3.2.2. El primero aporta estructuras de comandos y familias de tareas; el segundo proporciona las entidades con las que se construyen las instrucciones.

Sobre esta base se desarrolló una adaptación destinada a producir pares de órdenes y objetivos canónicos. Se incorporaron mecanismos para conservar el contexto entre subtareas, generar las etiquetas semánticas, introducir variantes lingüísticas y controlar la distribución y calidad de las muestras. De esta forma, la herramienta de generación se integró en un procedimiento de construcción de datos para la especialización del intérprete.

La separación entre estructuras de tareas y catálogos se conservó en la adaptación. Esto permite modificar las entidades de una instancia sin alterar necesariamente la composición de sus objetivos. El alcance lingüístico se mantiene conforme a la Sección 3.2.3: las plantillas sirven para construir datos controlados, mientras que el sistema busca interpretar otras formulaciones que expresen intenciones pertenecientes al mismo dominio.

4.2.2. Catálogos de personas, objetos y ubicaciones

El conocimiento K , definido en la Sección 3.3.3, se concretó mediante catálogos de nombres de personas, objetos, categorías, habitaciones y ubicaciones. Se incluyeron formas singulares y plurales de categorías, así como las descripciones admitidas para búsquedas de personas y selección de objetos.

Se implementó una validación de los catálogos para detectar condiciones que comprometen la generación de etiquetas, como un catálogo esencial vacío o una entidad clasificada simultáneamente como persona y objeto. También se incorporó una comparación de nombres con el banco de conocimiento de Justina para identificar diferencias entre la generación de datos y el sistema integrado.

La clasificación de entidades se apoya en el tipo y en el contexto semántico. Se evitó utilizar la capitalización como criterio único, pues un objeto puede tener un nombre escrito con mayúscula inicial. Además, se distinguió conceptualmente una ubicación de navegación de un soporte de colocación: la primera indica dónde debe desarrollarse una subtarea; el segundo especifica una superficie o recipiente relacionado con la manipulación.

Para la evaluación comparativa, la versión de los catálogos forma parte de la configuración que se mantendrá fija. Este control incluye su uso en el generador, en el adaptador simbólico y en las instrucciones proporcionadas a GPT.

4.2.3. Familias de tareas

El generador adaptado contempla 29 familias de comandos, organizadas en dos categorías principales: tareas de personas y tareas de objetos. Estas categorías sirven para distribuir las muestras según la familia generadora. Una tarea de objetos puede incluir un destinatario humano, por lo que la clasificación no exige que todas las entidades de la instrucción pertenezcan a un mismo tipo.

Se incluyeron tareas de navegación, búsqueda, saludo, seguimiento, guía, conteo, adquisición y comunicación de información, y transporte de objetos. Durante la adaptación se añadieron familias simples para buscar una persona, buscar un objeto y saludar a una persona en una habitación. Su incorporación permite representar estas intenciones como órdenes completas, además de su aparición dentro de instrucciones compuestas.

Para distinguir las composiciones alcanzables dentro de una familia se utilizó la firma de objetivos: una secuencia que conserva los nombres de los objetivos y omite sus argumentos concretos. Por ejemplo, [go, find] identifica una búsqueda precedida de navegación. Las firmas permiten distribuir muestras según la estructura de la tarea y analizar diferencias que la clasificación por familia, por sí sola, puede ocultar.

4.2.4. Representación de los objetivos de referencia

La representación de referencia adoptada para la comparación utiliza el vocabulario canónico de la Tabla 3.1 y conserva el orden de los objetivos. Para construir las etiquetas del conjunto de datos se desarrollaron funciones semánticas asociadas con las familias de comandos. Estas funciones transforman las entidades y relaciones del contexto de generación en una secuencia de objetivos.

La construcción es programática: la etiqueta se obtiene del contexto con el que se produce la instrucción, sin solicitar una interpretación al modelo que será entrenado. En las tareas compuestas, las funciones propagan el objeto o la persona de referencia hacia las subtareas posteriores. Los objetivos **find** y **count** incluyen el tipo mediante **kind**, mientras que los argumentos conservan los destinos, destinatarios y atributos relevantes.

Una misma función semántica puede utilizarse para diferentes formulaciones de una tarea. Por ejemplo, la familia de transporte entre ubicaciones construye la firma [go, find, take, place] a partir del objeto, el origen y el destino seleccionados. Esta organización vincula la orden con su etiqueta y reduce la necesidad de definir una anotación independiente para cada realización superficial.

Por ejemplo, la instrucción:

“Navigate to the bed then find a chocolate jello and take it and give it to me.”

se asocia con la siguiente representación:

```
go(bed)  
find(chocolate jello, kind=object)  
take(chocolate jello)  
deliver(chocolate jello, to=me)
```

En este caso, las referencias *it* mantienen la identidad del objeto introducido en la búsqueda y *me* identifica al operador como destinatario. La etiqueta recoge las sub-tareas de la orden, mientras que su expansión determinará las acciones concretas de navegación, percepción, manipulación e interacción.

Durante el desarrollo se realizó una revisión manual inicial de algunos pares de instrucciones y objetivos. Posteriormente se incorporaron comprobaciones automatizadas sobre las etiquetas y las propiedades de generación. La revisión manual fue parcial y no constituyó una anotación exhaustiva ni una evaluación independiente de todos los ejemplos.

Las etiquetas se utilizan como referencias de origen programático. Su consistencia se examina mediante los controles de la Sección 4.3.7, que incluyen expectativas definidas en las pruebas para determinadas estructuras. Estos controles detectan errores de contrato y de composición, pero no sustituyen una comparación semántica independiente entre cada instrucción y su etiqueta. Un error compartido por la función generadora y sus controles puede permanecer en el conjunto y propagarse a varias muestras.

Las intenciones de comunicación de información se representan mediante **tell** y las de colocación mediante **place**, conforme al vocabulario de la Tabla 3.1. Se mantiene la distinción entre objetivos y acciones ejecutables: la expansión de **place** puede incluir una acción **drop** del ejecutor.

4.3. Construcción del conjunto de datos

4.3.1. Adaptación del CommandGenerator

La adaptación se diseñó para vincular la generación lingüística con la generación semántica mediante un contexto compartido. En ese contexto se conservan las entidades seleccionadas y las relaciones necesarias para resolver las continuaciones de una orden. La decisión evita que la frase y su etiqueta se construyan mediante selecciones independientes que puedan producir referencias incompatibles.

Se separó la definición de las formas de expresar una tarea de la función que determina sus objetivos. Las variantes superficiales controlan la redacción, mientras que las funciones semánticas establecen la secuencia canónica. Con esta organización, añadir una formulación para una intención existente no requiere modificar su representación semántica; incorporar una tarea diferente exige definir también su composición de objetivos.

Se introdujeron además ajustes para conservar información necesaria en tareas compuestas. Las órdenes que solicitan el nombre, la postura o el gesto de una persona incluyen la adquisición de esa información mediante **save** antes de comunicarla con **tell**. En el transporte entre ubicaciones se utilizó **place** con la relación **at** para indicar el destino sin imponer una relación de apoyo sobre una superficie. También se corrigió la clasificación de una familia de adquisición de objetos para asignarla a la categoría correspondiente.

Estos cambios forman parte del desarrollo del conjunto de datos y buscan preservar la correspondencia entre intención, etiqueta y expansión. La estructura de los archivos y las funciones que los implementan se presenta en el capítulo 5.

La Tabla 4.1 sintetiza las decisiones de adaptación y su función dentro de la metodología. Se distingue la construcción de etiquetas de los mecanismos que controlan su estructura y la distribución de las muestras.

4.3.2. Generación de órdenes

La construcción de una muestra comienza con la selección de una categoría, una familia y una firma de objetivos alcanzable. A continuación se determinan las subtareas que componen la orden, se seleccionan las entidades compatibles y se sustituyen los elementos variables de la plantilla. El contexto se propaga entre las subtareas para conservar sus referencias.

Cada muestra contiene la instrucción, los objetivos y metadatos que identifican su procedencia. Se registran la familia, la categoría, la variante superficial, la firma de objetivos, los tipos de entidades y los campos utilizados. Esta información permite analizar la cobertura del conjunto y agrupar los errores por características de la tarea.

Se incorporaron una semilla de generación y un límite de reintentos para controlar el procedimiento. La muestra se acepta después de comprobar su etiqueta y su condición de unicidad. Cuando una combinación no cumple los criterios, se realiza un nuevo intento dentro de los límites establecidos. La reproducción del conjunto depende de conservar,

Tabla 4.1: Modificaciones incorporadas en la adaptación del generador.

Aspecto	Modificación	Propósito
Construcción de etiquetas	Funciones semánticas asociadas con familias y subtareas.	Obtener objetivos a partir del contexto utilizado para construir la orden.
Contexto compartido	Propagación del objeto o la persona entre subtareas relacionadas.	Mantener la identidad de las entidades durante la composición de la tarea.
Variación lingüística	Separación de las plantillas superficiales y la generación semántica.	Variar la formulación conservando la intención y sus argumentos.
Cobertura de tareas	Incorporación de familias simples de búsqueda y saludo.	Representar esas intenciones como órdenes completas, además de subtareas.
Composición semántica	Captura de información con save y representación del destino de transporte mediante at .	Conservar operaciones y relaciones necesarias antes de informar o colocar.
Distribución y control	Balance por categoría, familia y firma; deduplicación y validación.	Controlar la cobertura, evitar repeticiones y detectar inconsistencias.

además de la semilla, las mismas versiones del generador y de sus catálogos.

4.3.3. Generación de objetivos canónicos

Las funciones semánticas transforman el contexto de la muestra en una secuencia de objetivos. En las instrucciones compuestas, esta transformación conserva los objetos y personas compartidos entre acciones y establece el orden lógico de las operaciones. Las intenciones de colocación se representan mediante **place** y las de comunicación de información mediante **tell**, de acuerdo con el vocabulario común de la investigación.

Se desarrolló un validador del contrato semántico que comprueba los objetivos permitidos, sus argumentos y las relaciones admitidas. También verifica dependencias como la adquisición de un objeto antes de su entrega o colocación. Estas comprobaciones permiten rechazar etiquetas incompletas o incompatibles antes de incorporarlas al conjunto.

Para comparar representaciones se distingue la escritura de las entidades de su contenido semántico. La normalización de mayúsculas puede utilizarse cuando preserve la identidad establecida por el catálogo. En todos los casos se mantienen el tipo, los valores de los argumentos, las relaciones y el orden de los objetivos, de modo que una

normalización superficial no oculte una diferencia de significado.

4.3.4. Variación lingüística

Se incorporaron varias realizaciones superficiales por familia para introducir distintas formas de expresar una misma estructura de tarea. Las variantes utilizan los mismos campos de contenido y comparten la función de generación semántica. Esta decisión permite variar la forma lingüística sin cambiar la etiqueta cuando se conservan la intención y las entidades.

Se implementó una comprobación de los campos de las variantes al iniciar la generación. Su finalidad es detectar omisiones o incorporaciones de información incompatibles con la familia. La conservación de esos campos no equivale a una demostración de equivalencia lingüística: el orden, el alcance de las expresiones y las relaciones entre subtareas también determinan el significado de la instrucción.

La diversidad introducida mediante plantillas se considera variación controlada dentro del dominio. Para estudiar formulaciones que no procedan de esas plantillas, el diseño de evaluación contempla un grupo separado de reformulaciones elaboradas de manera independiente del entrenamiento. Esta distinción permitirá analizar por separado la interpretación de estructuras conocidas y la respuesta ante nuevas formas de expresar las mismas tareas.

4.3.5. Balance y deduplicación

El conjunto de datos se construyó con 40 000 muestras. Para distribuir la generación se asignó la mitad a familias de personas y la mitad a familias de objetos. Dentro de cada categoría se establecieron cuotas por familia y por firma de objetivos, con el propósito de reducir la concentración de ejemplos en las estructuras con mayor probabilidad de ser seleccionadas.

La deduplicación se realizó sobre la instrucción convertida a minúsculas y con espacios normalizados. Cuando una misma entrada aparecía asociada con objetivos diferentes, el procedimiento la identificaba como un conflicto de etiqueta y la rechazaba. Este criterio evita repeticiones textuales normalizadas y etiquetas contradictorias para una misma entrada, aunque no agrupa todas las expresiones semánticamente equivalentes.

Se incorporó una redistribución de cuotas para las familias cuyo número de combinaciones únicas resulta limitado. Al alcanzar el límite de reintentos, las muestras

faltantes se reasignan dentro de la misma categoría, dando prioridad a las familias y firmas menos representadas. Así se conserva la distribución entre tareas de personas y objetos sin introducir duplicados para completar una cuota.

El balance resultante debe interpretarse en estos dos niveles: la proporción entre categorías se mantiene, mientras que la distribución entre familias puede variar debido a la saturación. Las cantidades finales por familia y firma se presentan junto con las características del conjunto en el capítulo 5.

4.3.6. División de entrenamiento y evaluación

Las 40 000 muestras se dividieron en 36 000 ejemplos de entrenamiento y 4 000 de validación. La partición de entrenamiento se utilizó para ajustar los adaptadores del modelo y la de validación para observar su comportamiento durante el ajuste. Esta última forma parte del desarrollo del intérprete y se distingue del conjunto destinado a la comparación final de arquitecturas.

La eliminación de duplicados exactos no impide que ambas particiones compartan plantillas o contengan formulaciones de una misma intención. Por ello, los resultados sobre esta división describen el comportamiento del modelo bajo la distribución generada y no bastan para establecer su capacidad de generalización a formulaciones independientes.

Para la evaluación comparativa se reserva como criterio de diseño un conjunto que no intervenga en el entrenamiento, la selección del modelo ni la modificación de las reglas o del prompt. Cuando se estudien grupos de reformulaciones de una intención concreta, se mantendrá identificada su relación y se controlará su separación respecto de los datos de desarrollo. El procedimiento de construcción y las condiciones de independencia de esa prueba se especifican en el capítulo 6.

Durante el desarrollo se utilizaron además 48 comandos para comprobar el funcionamiento del sistema. Estos casos constituyen una prueba funcional y se mantienen separados de la evaluación experimental. Su descripción corresponde a la verificación de la implementación.

4.3.7. Validación previa del conjunto de datos

Después de la revisión manual parcial se desarrollaron comprobaciones automatizadas para examinar el contrato de las etiquetas, las propiedades del generador y su

compatibilidad con CLIPS. El procedimiento distingue la validación de los catálogos, los controles sobre las secuencias de objetivos y la comprobación de expansión. Cada nivel aporta evidencia sobre una propiedad específica del conjunto.

El validador de objetivos comprueba la sintaxis, los nombres y argumentos admitidos, la presencia de destinos obligatorios y el tipo explícito en **find** y **count**. También detecta calificadores incompatibles con el tipo declarado y algunos conflictos de tipo entre subtareas. En la secuencia se verifica que una entrega o colocación esté precedida por la adquisición del objeto y se detectan ciertas navegaciones o destinos redundantes. Estos controles operan sobre los objetivos; no interpretan de nuevo la instrucción para contrastar su significado.

Se incorporaron además pruebas sobre propiedades de generación, como la cardinalidad solicitada, la ausencia de entradas textuales duplicadas, la proporción entre categorías y la presencia de metadatos. Algunas pruebas contienen expectativas específicas: las familias de búsqueda simple deben producir la firma [go, find], mientras que la de saludo simple debe producir [go, find, greet]. Otras comprueban la presencia de **save** y del retorno al punto de instrucción en las tareas de información, o el uso de **at** en el transporte entre ubicaciones. Estas expectativas están codificadas como propiedades y firmas; no constituyen un conjunto completo de pares anotados de manera independiente.

La comprobación con CLIPS se realizó sobre un subconjunto de secuencias. Para cada una, el validador reinicia el entorno, convierte los objetivos en hechos y ejecuta las reglas. Se comprueba que no queden objetivos pendientes, que no se emita la respuesta correspondiente a un objetivo no soportado y que se produzca la señal de finalización **GPSR_DONE**. La aceptación indica que la secuencia satisface estas condiciones de expansión, sin acreditar por sí sola el cumplimiento semántico o físico de la tarea.

Se implementaron dos recorridos de comprobación. En la herramienta de auditoría, las secuencias válidas se procesan en el orden de lectura hasta alcanzar el límite configurado. En las pruebas del generador se selecciona el primer ejemplo disponible de cada familia del lote de prueba. Estas selecciones ofrecen coberturas diferentes y no equivalen por sí mismas a un muestreo aleatorio del conjunto completo. La validación efectuada sobre algunas muestras se distingue de una verificación exhaustiva de las 40 000 entradas.

Los metadatos asociados con cada muestra permiten identificar la familia, la categoría, la variante superficial, la firma, los tipos de entidades y los campos presentes.

Se conservan en el conjunto de datos y se omiten del contenido utilizado para entrenar el intérprete. Su función es describir la procedencia y estructura de la muestra; no almacenan el resultado individual de la comprobación con CLIPS ni acreditan que esa muestra haya sido examinada por todos los validadores.

La herramienta de auditoría calcula por separado los recuentos de etiquetas válidas e inválidas, las incidencias, la cobertura y los resultados de las secuencias comprobadas con CLIPS, y presenta un reporte agregado. Así se distingue la información descriptiva del conjunto de la evidencia de una ejecución concreta de validación.

El alcance de estos controles es la consistencia respecto de restricciones programadas y la compatibilidad de expansión de las secuencias examinadas. La fidelidad de la etiqueta permanece definida por su correspondencia con la orden. Para la evaluación final, las referencias y los criterios de inclusión se establecerán a partir de la intención y de las capacidades declaradas, sin exigir previamente que CLIPS resuelva cada caso. Esta separación permite analizar los errores compartidos por generación y validación y conservar los casos de falta de cobertura simbólica.

4.4. Diseño de la arquitectura Qwen–CLIPS

4.4.1. Función asignada a Qwen

Se asignó a Qwen2.5-7B la transformación de la instrucción x en una secuencia $G_{\text{pred}}^{\text{Qwen}}$, expresada mediante la representación canónica definida en la Sección 3.3.4. Esta delimitación permite especializar el modelo en la identificación de subtarefas, entidades y relaciones, y mantener la construcción de acciones en el componente simbólico.

La respuesta del intérprete se restringió a una estructura que contiene la secuencia de objetivos. La decisión facilita su procesamiento automático y establece un punto de observación entre el lenguaje natural y la planificación. Gracias a esta interfaz, es posible comprobar la interpretación antes de examinar el plan y localizar qué información recibe efectivamente CLIPS.

4.4.2. Especialización para interpretación semántica

La especialización se realizó mediante aprendizaje supervisado con los pares de instrucciones y objetivos contruidos para el dominio. Cada entrada se presentó al modelo como contexto y la secuencia canónica como respuesta esperada. Los metadatos de ge-

neración se conservaron para el análisis del conjunto y no se incluyeron en la respuesta que el modelo aprende a producir.

Para concentrar el ajuste en la generación de los objetivos, se calculó la pérdida sobre los tokens de la respuesta, excluyendo las posiciones del prompt y del relleno. Se utilizó QLoRA con el modelo base cuantizado a cuatro bits y adaptadores entrenables, lo que permitió especializarlo dentro de los recursos de cómputo disponibles.

Durante el desarrollo se exploró inicialmente Qwen2.5-0.5B. Su desempeño resultó insuficiente para las órdenes complejas consideradas y se continuó con Qwen2.5-7B, cuyo tamaño fue el mayor viable bajo los recursos locales disponibles. Esta exploración forma parte de las decisiones de desarrollo; la comparación experimental de la tesis se concentra en la configuración Qwen-CLIPS seleccionada y en GPT directo.

Los hiperparámetros, la configuración de inferencia y la identificación del adaptador utilizado se presentan en el capítulo 5, junto con el procedimiento de entrenamiento correspondiente.

4.4.3. Interfaz entre objetivos y planificación simbólica

Se desarrolló un adaptador para convertir la salida estructurada del intérprete en hechos que puedan procesarse mediante CLIPS. La interfaz se diseñó para conservar el orden, el nombre del objetivo, el tipo de entidad y los argumentos que determinan su expansión. La normalización de escritura establece la correspondencia entre los nombres de la representación semántica y los identificadores utilizados por el ejecutor.

El adaptador incorpora inferencias de tipo y reparaciones para determinadas secuencias. Estas operaciones forman parte del sistema integrado y pueden modificar la predicción original. Por ello, el diseño de evaluación distingue la salida directa de Qwen de la representación que finalmente se entrega al motor simbólico. Un objetivo corregido por el adaptador puede mejorar el resultado del sistema sin constituir un acierto del modelo antes de esa intervención.

La preservación de argumentos constituye un criterio de validez de la interfaz. Una pérdida de destino o de un atributo relevante durante la conversión se clasifica como un error de integración. Esta separación permite evitar que un problema de representación entre componentes se atribuya automáticamente a la interpretación o a las reglas.

4.4.4. Función asignada a CLIPS

Se desarrollaron reglas de producción para expandir los objetivos en acciones compatibles con Justina. La selección de la siguiente meta se restringió por su posición en la secuencia, de modo que las reglas procesen los objetivos en el orden recibido. Esta organización permite expresar explícitamente el conocimiento operativo de cada tipo de tarea.

Las reglas utilizan el estado esperado descrito en la Sección 3.4.2 para considerar los efectos previstos de las acciones anteriores. Se mantiene información de ubicación y posesión de objetos, además de referencias a personas y análisis previos. Con ello se comprueban algunas dependencias y se reutiliza el contexto establecido durante la construcción del plan.

Se incorporaron también convenciones de interacción y cierre, como los anuncios, el control del foco y el regreso al punto de instrucción. Estas operaciones complementan la realización de las tareas dentro del funcionamiento de Justina y forman parte de la especificación común proporcionada a la arquitectura directa.

La expansión es determinista para una misma base de reglas, hechos iniciales y configuración. Su finalización indica que concluyó la generación de acciones, mientras que el cumplimiento de la orden se evalúa mediante los criterios de validez del plan. En particular, una respuesta hablada que informe de la imposibilidad de ejecutar un objetivo se considera falta de cobertura de esa subtarea.

4.5. Diseño de la arquitectura GPT directo

4.5.1. Funciones asignadas al modelo

Se diseñó una arquitectura en la que un modelo GPT recibe la instrucción x y produce los objetivos $G_{\text{pred}}^{\text{GPT}}$ y el plan P^{GPT} durante una misma inferencia. La decisión de conservar ambos productos permite aplicar criterios de interpretación comparables a los de Qwen y examinar la coherencia entre los objetivos declarados y las acciones propuestas.

La generación se condiciona mediante una especificación operativa detallada, elaborada a partir del dominio y de las convenciones de expansión utilizadas en CLIPS. Por tanto, la condición directa estudia la capacidad del modelo para generar un plan ajustado a esa especificación. Su alcance corresponde a las tareas, restricciones y con-

figuraciones de esta investigación.

Los objetivos de GPT se conservan como producto de análisis y no se envían a CLIPS. La evaluación distingue los dos artefactos de la respuesta, aunque su producción no constituya dos componentes independientes como en Qwen-CLIPS.

4.5.2. Conocimiento y restricciones proporcionadas

Para orientar la generación se elaboró un prompt con los catálogos de entidades, el vocabulario de objetivos, las acciones disponibles y las restricciones de composición. Se incluyeron criterios sobre resolución de referencias, dependencias de manipulación, relaciones espaciales e interacción con personas.

La especificación también establece cómo conservar coherencia con el estado esperado durante la generación. El modelo recibe instrucciones para considerar la ubicación, el objeto sostenido y las entidades identificadas al determinar las acciones posteriores. Se incorporaron las convenciones de identificación, inicio y finalización del plan para facilitar su compatibilidad con el ejecutor.

El conocimiento operativo se presenta así mediante instrucciones en la arquitectura directa y mediante reglas en el componente CLIPS. Para la comparación se fijarán la versión del modelo, el prompt y sus configuraciones de generación, evitando modificar estas condiciones a partir de los resultados de la prueba final.

4.5.3. Contrato de salida

Se definió un contrato de respuesta que contiene el estado de la solicitud, un mensaje, los objetivos y el plan. Para los casos ejecutables se establece una secuencia ordenada de pasos con los campos requeridos por el ejecutor, acompañada por las señales de inicio y finalización. La estructura facilita la extracción y el análisis automático de los productos del modelo.

El diseño utiliza un esquema de salida para restringir los campos, tipos y nombres de acciones admitidos. La validación del contenido se considera una comprobación adicional: además de cumplir la estructura, la respuesta debe conservar las entidades de la orden, respetar las dependencias y cubrir las subtarefas solicitadas.

La evaluación conservará la respuesta original y distinguirá los errores de formato, las inconsistencias de contenido y los fallos de comunicación con el servicio. Esta clasificación permite analizar los resultados del modelo sin confundirlos con problemas de

transporte o procesamiento de su respuesta.

4.5.4. Plan ejecutable, aclaración y rechazo

Se adoptaron los estados `executable`, `clarification` y `rejected`, definidos en la Sección 3.4.3. Esta decisión permite representar tanto la producción de un plan como las situaciones en las que falta información o la tarea excede las restricciones del dominio. Las respuestas de aclaración y rechazo contienen un mensaje y no incluyen acciones operativas.

La comparación principal se delimita a instrucciones resolubles dentro del dominio común. En estas condiciones, una aclaración innecesaria o el rechazo de una orden válida se consideran ausencia de una respuesta ejecutable correcta y se mantienen en el total de casos evaluados.

El comportamiento ante órdenes ambiguas o fuera del dominio se considera un análisis adicional. Su inclusión requiere criterios específicos para las respuestas disponibles en cada arquitectura, sin suponer que el rechazo de una salida inválida en Qwen-CLIPS equivale a un mecanismo conversacional de aclaración.

4.6. Diseño de la comparación

4.6.1. Igualdad del dominio de evaluación

La comparación se plantea sobre el mismo conjunto de instrucciones y referencias, con las condiciones comunes establecidas en la Sección 4.1. Cada caso se identificará por su familia, estructura de objetivos y procedencia lingüística. La conservación de esta información permitirá examinar diferencias por tipo de tarea y por formulación.

Las referencias se definirán a partir de la intención de la orden y del dominio de capacidades. Los ejemplos utilizados para modificar el generador, el modelo, las reglas o el prompt permanecerán identificados como parte del desarrollo. El procedimiento experimental establecerá el conjunto de prueba y lo mantendrá fijo durante la comparación.

Se registrará también la entrada efectiva de cada arquitectura. La normalización previa puede modificar aspectos superficiales o resolver ciertas variaciones antes de la inferencia, por lo que forma parte de la configuración evaluada. La medición de latencia

considerará el recorrido completo de la solicitud y distinguirá, cuando sea posible, el procesamiento del modelo de la comunicación y la expansión.

La ubicación del cómputo se tratará como una condición de despliegue. El uso de Qwen con pesos bajo control del proyecto permite alojar la inferencia en infraestructura propia, incluida una computadora remota. Por ello, su comparación temporal con GPT debe considerar las condiciones de comunicación utilizadas en cada caso.

4.6.2. Igualdad del repertorio de acciones

Los planes se evaluarán respecto de un contrato común de acciones, parámetros y efectos esperados. Se mantendrán las mismas capacidades de manipulación, convenciones de identificación y políticas de cierre. Este control permite comparar la realización de una tarea dentro de las posibilidades de Justina.

Se distinguen dos aspectos del resultado: la fidelidad a las convenciones operativas y el cumplimiento de la instrucción. La correspondencia con una expansión de referencia permite estudiar la reproducción de esas convenciones. En GPT directo, este aspecto refleja también el seguimiento de la especificación incluida en el prompt.

El cumplimiento se examinará mediante la cobertura de subtarear, el orden de las acciones, sus parámetros y su coherencia con el estado esperado. Una diferencia en el texto de un anuncio puede conservar el significado de la tarea, mientras que una secuencia idéntica a una expansión de referencia puede resultar insuficiente si omite una operación necesaria. Por ello, la coincidencia con CLIPS no se utiliza como criterio único de corrección.

La correspondencia entre objetivos y capacidades se establece a partir de los efectos que las acciones permiten obtener. Las expresiones habladas sobre una tarea no sustituyen la operación necesaria para realizarla. Este criterio se aplica también a los atributos y propiedades solicitados, cuya conservación debe examinarse en la traducción hacia las acciones del ejecutor.

4.6.3. Separación entre interpretación y planificación

La interpretación se evaluará mediante la correspondencia de $G_{\text{pred}}^{\text{Qwen}}$ y $G_{\text{pred}}^{\text{GPT}}$ con la misma secuencia de referencia G_{ref} . Para la planificación se examinarán los planes $P^{\text{Qwen-CLIPS}}$ y P^{GPT} , considerando tanto su coherencia con los objetivos producidos por cada arquitectura como su cumplimiento de la tarea de referencia. Esta doble

relación permite identificar un plan internamente coherente que, sin embargo, resuelve una interpretación equivocada de la instrucción.

En Qwen-CLIPS, el análisis distinguirá la predicción del modelo, su transformación por el adaptador y el plan generado. La condición de control utilizará los objetivos de referencia bajo la misma configuración simbólica. Para estudiar la cadena integrada se considerará también el recorrido de conversión empleado, de manera que las diferencias entre una comprobación fuera de línea y la operación en ROS no se atribuyan directamente a las reglas.

En GPT directo se analizarán los objetivos y las acciones de una misma respuesta. La clasificación permite describir qué producto incumple los criterios de validez, aunque no identifica con certeza una etapa causal interna del modelo. En ambas arquitecturas se conservarán los casos inválidos y los fallos de interfaz como parte del registro experimental.

Con esta organización, el diseño relaciona la construcción de datos, la especialización del intérprete y la generación de planes con criterios de evaluación diferenciados. El capítulo siguiente presenta la implementación de los componentes desarrollados y su integración con Justina.

Capítulo 5

Implementación

Este capítulo describe los componentes desarrollados para transformar instrucciones en lenguaje natural en planes de acciones para Justina. La implementación concreta las decisiones metodológicas del Capítulo 4 mediante la generación de datos supervisados, la especialización del intérprete Qwen, la conversión de objetivos a hechos simbólicos y la expansión de acciones con CLIPS.

El sistema se organiza en componentes con responsabilidades diferenciadas. La construcción del conjunto de datos y el entrenamiento se realizan fuera del ciclo de operación del robot. Durante la planificación, el intérprete recibe una instrucción y devuelve objetivos que se validan y convierten antes de ingresar al motor simbólico. El plan resultante se publica para su consumo por el sistema de ejecución. La arquitectura GPT directo se trata como una implementación separada, con su propio contrato de respuesta e interfaz de generación.

Como ejemplo conductor se utiliza la orden de búsqueda, adquisición y entrega de un objeto introducida en el capítulo anterior. Sus diferentes representaciones permiten relacionar la entrada lingüística, los objetivos, los hechos CLIPS y las acciones del ejecutor. Los resultados cuantitativos de entrenamiento y evaluación se reservan para el capítulo de resultados.

5.1. Plataforma experimental

5.1.1. Robot Justina

La implementación se integra en la arquitectura ViRBot de Justina, descrita en la Sección 3.1.1. El sistema de planificación utiliza las capacidades de navegación, percepción, manipulación e interacción disponibles mediante las interfaces del robot. Su salida especifica las acciones que deben solicitarse a esos componentes.

La comunicación con la plataforma se realiza mediante ROS 2. El planificador produce identificadores de acciones y parámetros simbólicos, como una ubicación de navegación, un objeto o una referencia a una persona. La resolución física de esas instrucciones corresponde al ejecutor y a los módulos encargados de cada tarea.

En el ciclo GPSR se utiliza `instruction_point` para representar el punto de interacción con el operador. Este símbolo se distingue de `inspection point`, que puede aparecer como una entidad del catálogo. La diferencia se conserva al normalizar los parámetros del plan.

5.1.2. Plataforma de cómputo

Para el trabajo con Qwen2.5-7B se utilizó una computadora con una GPU ASUS RTX 4070 Ti, un procesador Intel Core i7 de undécima generación y 32 GB de memoria RAM. La adaptación del modelo se configuró considerando los recursos disponibles en este equipo.

La implementación admite dos modalidades de despliegue de la inferencia: local y remota. En la modalidad local, Qwen se ejecuta en el mismo equipo que controla al robot y comparte sus recursos de cómputo con los módulos de navegación, percepción e interacción. Esta modalidad permite realizar la interpretación sin recurrir a un servidor externo, siempre que el equipo disponga de memoria y capacidad de procesamiento suficientes para las tareas concurrentes.

Cuando la carga conjunta exige liberar recursos del equipo del robot, el modelo puede desplegarse como un servicio en otra computadora con GPU, accesible mediante una red local o a través de Internet. En esta modalidad, Justina envía la orden al servidor por HTTP y recibe los objetivos semánticos generados. El modelo permanece cargado en el servidor entre solicitudes, mientras que la conversión a hechos, la expansión simbólica y la comunicación con el ejecutor se realizan en el entorno ROS 2 del robot.

Ambas modalidades conservan la misma función del intérprete y el mismo contrato de objetivos. La elección depende de los recursos disponibles y de la carga de los demás componentes. El despliegue remoto permite trasladar el cómputo de inferencia a otro equipo e incorpora el tiempo de comunicación de red al recorrido de la solicitud. Por ello, la modalidad utilizada y la ubicación de los procesos forman parte de la configuración experimental al medir latencia y uso de recursos.

5.1.3. Entorno de software y ROS 2

Los componentes de generación, validación e interpretación se desarrollaron en Python. El entrenamiento utiliza PyTorch para el cálculo de tensores, Transformers para cargar Qwen y gestionar el ajuste mediante `Trainer`, PEFT para incorporar los adaptadores LoRA y bitsandbytes para cargar los pesos cuantizados. La preparación de las particiones utiliza la biblioteca Datasets. La interacción con CLIPS se implementó mediante `clipsy`, cuyo módulo Python se importa como `clips`.

ROS 2 proporciona los servicios y tópicos que conectan el intérprete, el adaptador y el planificador con Justina. Los archivos de lanzamiento agrupan los nodos y configuran sus nombres, parámetros y espacios de nombres. Esta organización permite ejecutar pruebas de componentes por separado y después utilizar el mismo contrato en el sistema integrado.

La reproducibilidad requiere identificar las versiones del sistema operativo, la distribución de ROS 2, Python y las bibliotecas de entrenamiento e inferencia. Estas versiones se vinculan con el entorno efectivamente utilizado, ya que las configuraciones de desarrollo y de despliegue pueden diferir.

5.2. Implementación del conjunto de datos

5.2.1. Generador de órdenes

La generación se organiza alrededor de `generate_dataset.py`, que coordina la selección de muestras, el balance, la deduplicación y la validación previa a la escritura. El componente `gpsr_commands.py` construye las órdenes utilizando las familias y variantes definidas en `command_constants.py`.

La generación semántica se implementó en `command_goals.py` mediante la clase `CommandGoalsMixin`. El método `_generate_goals` selecciona la función registrada para

una familia y le entrega el contexto de construcción. Cada función devuelve una lista ordenada de objetivos. Las continuaciones de una orden pueden invocar otras funciones y recibir el objeto o la persona establecidos por las subtareas anteriores.

Por ejemplo, la función de transporte entre dos ubicaciones toma el objeto, el origen y el destino del contexto y construye los objetivos de navegación, búsqueda, adquisición y colocación. La función de entrega al operador utiliza el objeto conservado en el contexto y genera `deliver(object, to=me)`. Esta propagación permite mantener el referente de un pronombre en las distintas partes de una orden.

La representación de entrada y etiqueta del ejemplo conductor es:

```
{
  "input": "Navigate to the bed then find a chocolate jello and take it and give it to me",
  "goals": [
    "go(bed)",
    "find(chocolate jello, kind=object)",
    "take(chocolate jello)",
    "deliver(chocolate jello, to=me)"
  ]
}
```

El ejemplo omite los metadatos para destacar la correspondencia entre entrada y objetivos. En el archivo JSONL, cada muestra completa se serializa en una sola línea.

5.2.2. Catálogos y CompetitionTemplate

El módulo `knowledge.py` carga los catálogos mediante `parse_data`. La estructura de entrada separa los nombres de personas, las ubicaciones, las habitaciones y los objetos en archivos de `CompetitionTemplate`. Las rutas relativas utilizadas son:

```
names/names.md
maps/location_names.md
maps/room_names.md
objects/objects.md
```

El conocimiento cargado se entrega al generador para sustituir los elementos variables de las plantillas. La validación de catálogos se encapsula en `catalog_validation.py`; incluye la detección de colisiones y diferencias con el catálogo de nombres del banco de conocimiento de Justina.

La localización del espacio de trabajo del robot puede configurarse con `JUSTINA_WS`. Esto permite vincular las comprobaciones con el repositorio utilizado en el despliegue sin incorporar rutas personales al contrato de generación.

5.2.3. Validadores

La validación de etiquetas se implementó en `goal_schema.py`. La función `parse_goal` separa el nombre del objetivo, los argumentos posicionales y los campos nombrados. El resultado se almacena en una estructura `ParsedGoal`, que permite consultar el objetivo principal y el tipo de entidad sin depender de su capitalización.

La función `validate_goals` aplica comprobaciones individuales y de secuencia. Detecta sintaxis inválida, ausencia de argumentos obligatorios, campos no admitidos, incompatibilidades entre calificadores y tipos, y algunas dependencias incumplidas. Las incidencias se representan mediante `ValidationIssue`, con un código estable, un mensaje y, cuando corresponde, la posición del objetivo.

La auditoría del conjunto se implementó en `dataset_evaluation.py`. Su función `evaluate_dataset_rows` recorre las filas, valida sus objetivos y acumula estadísticas por familia, firma, tipo y campos. El módulo también contiene `ClipsPlanValidator`, que carga las reglas una vez y reinicia el entorno para cada secuencia examinada.

El programa `evaluate_dataset.py` permite ejecutar esa auditoría sobre un JSONL. Los límites de lectura y de comprobación con CLIPS se configuran de manera independiente. Cuando se fija un límite de secuencias CLIPS, se toman las primeras etiquetas válidas en el orden de lectura hasta alcanzarlo. El reporte agregado se imprime al finalizar; las etiquetas inválidas o los fallos de expansión producen un código de salida de error.

Las pruebas de `test_dataset_quality.py` comprueban propiedades del contrato y del generador sobre un lote de 500 muestras. Incluyen cardinalidad, deduplicación, distribución entre categorías y firmas esperadas para familias concretas. La prueba de expansión utiliza el primer ejemplo de cada familia presente en ese lote. Este tamaño de prueba se distingue de la cantidad de muestras comprobadas en la auditoría del conjunto final.

5.2.4. Dataset final

El conjunto se almacena en `dataset_gpsr.jsonl`, con un objeto JSON por línea. Cada fila contiene `input`, `goals` y `meta`. Los objetivos corresponden al vocabulario de la Tabla 3.1. La división considerada comprende 36 000 muestras de entrenamiento y 4 000 de validación.

Los metadatos identifican la familia, la categoría, la variante superficial, la firma de

objetivos, los tipos de entidades y los campos presentes. Se conservan para caracterizar el conjunto, pero se excluyen del contenido supervisado del modelo. No contienen los resultados individuales de la validación con CLIPS.

El entrenamiento utiliza la instrucción como entrada y un objeto con la propiedad `goals` como respuesta esperada. El cargador conserva temporalmente la familia y la categoría como columnas auxiliares; estas columnas se eliminan al construir las entradas tokenizadas y no forman parte de la conversación supervisada.

Después de validar las filas, se aplica `train_test_split` con una proporción de validación de 0.1 y semilla 42. La partición se realiza aleatoriamente por filas, sin agrupación ni estratificación por familia o plantilla. Por ello, la pérdida de validación caracteriza ejemplos de la distribución del generador y no demuestra, por sí sola, generalización a familias o plantillas excluidas del entrenamiento.

5.3. Entrenamiento e inferencia de Qwen2.5-7B

5.3.1. Preparación del modelo

El entrenamiento se organiza en `N1-C1.py` y selecciona el repositorio `Qwen/Qwen2.5-7B` como modelo base. Cada fila se valida y se transforma en una conversación mediante `apply_chat_template` del tokenizador. El mensaje del usuario contiene la orden y la respuesta del asistente contiene los objetivos serializados como JSON. Esta preparación utiliza esos dos roles, sin añadir un mensaje de sistema al ejemplo supervisado.

El cálculo de la pérdida se concentra en la respuesta. Las posiciones del prompt reciben la etiqueta de exclusión `-100`, al igual que el relleno añadido durante la preparación de los lotes. El componente `CompletionOnlyCollator` realiza el relleno dinámico a la derecha, hasta un múltiplo de ocho posiciones, y ajusta las máscaras de atención y las etiquetas. Si el tokenizador carece de token de relleno, se utiliza su token de fin de secuencia.

La conversación completa se tokeniza con truncamiento y longitud máxima de 512 tokens. El prefijo del usuario se tokeniza por separado para determinar las posiciones que se excluyen de la pérdida. El límite comprende tanto la orden como la respuesta, por lo que puede afectar a los objetivos situados al final de una secuencia extensa.

5.3.2. Configuración QLoRA

El modelo base se carga con cuantización de cuatro bits en formato NF4 y doble cuantización. Sus pesos permanecen congelados y el ajuste se realiza sobre adaptadores LoRA insertados en las proyecciones de atención y del bloque de transformación del modelo.

Los módulos seleccionados son `q_proj`, `k_proj`, `v_proj`, `o_proj`, `gate_proj`, `up_proj` y `down_proj`. Se utiliza rango 32, factor de escala 64 y una probabilidad de descarte de 0.05. Esta configuración delimita la capacidad entrenable sin actualizar todos los parámetros del modelo base.

El cálculo utiliza BF16 cuando la GPU declara soporte para ese formato y FP16 en caso contrario. La carga selecciona la implementación de atención `eager`, desactiva la ventana deslizante y distribuye el modelo con `device_map='auto'`. Durante el ajuste se desactiva la caché y se habilita *gradient checkpointing* con `use_reentrant=False`, reduciendo las activaciones conservadas a cambio de recalcularlas durante la retropropagación. El script requiere CUDA y limita a 0.9 la fracción de memoria disponible para el asignador de PyTorch en el dispositivo configurado.

Al guardar el modelo se conserva el adaptador y su configuración, junto con el tokenizador. La inferencia requiere cargar estos artefactos sobre la misma versión del modelo base.

5.3.3. Hiperparámetros de entrenamiento

La Tabla 5.1 reúne la configuración de entrenamiento establecida en el script de especialización. La acumulación de gradientes permite procesar una muestra por GPU y actualizar los parámetros después de acumular ocho muestras.

Con 36 000 muestras de entrenamiento, un lote efectivo de ocho y dos épocas, la configuración corresponde a 9 000 actualizaciones. Este valor es una consecuencia de la configuración y no sustituye el registro de la ejecución efectivamente realizada.

El optimizador y la semilla del entrenamiento no se sobrescriben explícitamente en `TrainingArguments`; su valor efectivo depende de la configuración resuelta por la versión de Transformers utilizada. La semilla 42 de la partición identifica una operación distinta de las fuentes de aleatoriedad del ajuste.

Tabla 5.1: Configuración de entrenamiento de Qwen2.5-7B.

Parámetro	Valor
Longitud máxima de entrada y respuesta	512 tokens
Tamaño de lote por GPU	1
Acumulación de gradientes	8
Tamaño efectivo de lote, con una GPU	8
Número de épocas	2
Tasa de aprendizaje inicial	1×10^{-4}
Planificador de tasa de aprendizaje	Coseno
Pasos de calentamiento	150
Intervalo de evaluación	250 pasos
Intervalo de guardado	500 pasos
Intervalo de registro de pérdida	50 pasos
Rango / escala / descarte LoRA	32 / 64 / 0.05

5.3.4. Selección del checkpoint

La selección automática se configura mediante estas opciones del entrenador:

```
load_best_model_at_end=True
metric_for_best_model="eval_loss"
greater_is_better=False
```

Al terminar, el entrenador restaura el checkpoint seleccionado por menor pérdida de validación entre los candidatos que registra para ese propósito. La evaluación se programa cada 250 actualizaciones y el guardado cada 500; estos intervalos deben distinguirse al relacionar una pérdida con un archivo conservado. Se configura además `save_total_limit=2` para limitar la retención.

Después de `trainer.train()`, se guardan el adaptador seleccionado y el tokenizador en `nl2cd_qwen7b`. La curva de entrenamiento se construye a partir de `trainer.state.log_history`, utilizando los registros de pérdida de entrenamiento y validación, y se exporta como `loss_curve.png`.

El script incluye también una comprobación sobre las primeras 50 muestras de validación, o sobre las disponibles si fueran menos. Este límite pertenece al código de comprobación posterior al ajuste y no define el conjunto experimental de comparación. La pérdida de validación, la coincidencia de objetivos y la expansión con CLIPS aportan evidencias distintas y se interpretan por separado.

El artefacto destinado a inferencia se identifica de manera conjunta por el modelo base, el adaptador y el tokenizador. El nombre de un directorio de salida no basta para determinar qué checkpoint contiene; la selección debe quedar vinculada con su paso y configuración de entrenamiento.

5.3.5. Inferencia

El módulo `inference.py` carga `Qwen/Qwen2.5-7B` en NF4 con doble cuantización y añade los pesos mediante `PeftModel.from_pretrained`. El tokenizador se recupera del directorio del adaptador y el modelo se coloca en modo de evaluación. El dispositivo predeterminado es `cuda:0`; las opciones de ejecución permiten cambiar la distribución del modelo y establecer límites de memoria.

Antes de tokenizar, el recorrido de inferencia aplica el normalizador obtenido mediante `get_default_normalizer` del módulo `command_normalizer`. La orden resultante se introduce como mensaje de usuario y la plantilla añade el inicio de la respuesta del asistente. La entrada normalizada se devuelve junto con la predicción cuando se solicita `return_normalized=True`.

La generación utiliza `do_sample=False`, penalización de repetición de 1.05 y un máximo de 128 tokens nuevos. Las condiciones de parada incluyen el token de fin de secuencia y `<|im_end|>`, cuando está disponible. El script de entrenamiento utiliza un límite de 256 tokens en sus comprobaciones posteriores al ajuste; por ello, ambos recorridos no deben tratarse como configuraciones idénticas de inferencia.

Se retiran los tokens de entrada de la secuencia producida y se decodifica únicamente la continuación. La extracción localiza el primer fragmento con llaves balanceadas e intenta convertirlo mediante `json.loads`. Es un procedimiento de extracción seguido de validación, sin restricción gramatical durante la generación. Si la extracción o la validación falla, se devuelve un error y hasta 200 caracteres de la respuesta como información diagnóstica.

La salida se procesa para obtener el objeto JSON y se somete a la validación del contrato semántico. El resultado aceptado contiene una lista `goals`; las respuestas que incumplen el contrato se rechazan antes de entregarlas al siguiente componente. Esta comprobación detecta errores de estructura y restricciones programadas, sin acreditar por sí sola la fidelidad de la interpretación.

La predicción directa corresponde a $G_{\text{pred}}^{\text{Qwen}}$. Las normalizaciones o reparaciones posteriores se consideran operaciones adicionales del sistema y se distinguen del resultado

inicial al evaluar el intérprete.

5.3.6. Despliegue del modelo

El intérprete Qwen admite dos modalidades de uso: ejecución local y ejecución como servicio remoto. Ambas utilizan el modelo base, el adaptador LoRA y el tokenizador, y producen objetivos mediante el mismo procedimiento de inferencia. La elección determina dónde se realiza el cómputo y cómo se comunica la orden al intérprete.

En la modalidad local, el modelo se carga en el mismo equipo que ejecuta los componentes de control del robot. El módulo `inference.py` proporciona las funciones de carga y traducción, por lo que la interpretación puede realizarse directamente desde un proceso Python. Una vez disponibles los archivos del modelo y del adaptador, el procesamiento de cada orden no requiere consultar un servidor externo. Esta modalidad comparte los recursos de CPU, GPU y memoria con los demás componentes de Justina.

Cuando la carga de trabajo requiere liberar recursos del equipo del robot, la inferencia puede trasladarse a otra computadora y exponerse como un servicio accesible mediante una red local o Internet. Para esta modalidad se desarrolló `server.py`, que mantiene cargados el modelo, el tokenizador y el normalizador entre solicitudes. Justina envía la instrucción por HTTP y recibe la predicción, mientras que la conversión a hechos y la planificación con CLIPS permanecen en el entorno ROS 2 del robot.

El servidor utiliza `ThreadingHTTPServer` y escucha por defecto en el puerto 8008. Aunque las solicitudes se atienden mediante hilos, un mecanismo de exclusión mutua permite una sola traducción a la vez. Esto evita que varias solicitudes ejecuten simultáneamente la generación sobre el mismo modelo.

La interfaz ofrece dos rutas: `GET /health`, para consultar la disponibilidad del servicio, y `POST /translate`, para solicitar la interpretación de una orden. La petición de traducción contiene un objeto JSON con la propiedad `command`:

```
{"command": "go to the kitchen"}
```

Cuando la predicción es aceptada, la respuesta incluye `ok=true`, el tiempo `elapsed_ms` y un objeto `result`. Dentro de este objeto, `normalized_input` conserva la entrada procesada y `prediction` contiene la lista `goals`. La envoltura HTTP se retira antes de entregar al adaptador el contrato semántico:

```
{"goals": ["go(kitchen)"]}
```

El servidor limita el cuerpo de la petición a 8192 bytes por defecto. Una traducción aceptada devuelve el código 200; una petición incorrecta o una orden vacía, 400; una predicción inválida, 422; y un fallo interno, 500. Cuando se configura una clave de acceso, el servidor comprueba el encabezado **Authorization** mediante el esquema **Bearer** y devuelve 401 si la clave no coincide.

La modalidad de despliegue también determina el alcance de las mediciones temporales. En el servicio remoto, `elapsed.ms` incluye la espera por el acceso al modelo, la normalización, la inferencia y el procesamiento de la salida. Excluye la carga inicial del modelo y el tiempo completo de comunicación observado por el cliente. Por ello, la evaluación distingue el tiempo de interpretación del tiempo total de la solicitud y registra si el modelo se ejecutó localmente o mediante el servicio remoto.

5.4. Implementación del planificador CLIPS

5.4.1. Conversión de objetivos a hechos `gpsr-goal`

El adaptador `goals_to_clips_node.py` recibe la lista de objetivos y construye hechos `gpsr-goal`. El componente `GoalParser` acepta un objeto con la propiedad `goals` o directamente una lista JSON. Analiza cada expresión y separa los argumentos posicionales de los pares `clave=valor`, respetando las comas delimitadoras y los valores entre comillas. Cada hecho conserva la posición, el nombre del objetivo, el objetivo principal y los campos utilizados para su expansión. La estructura compartida es:

```
(deftemplate gpsr-goal
  (slot step (type INTEGER))
  (slot name)
  (slot target (default ""))
  (slot kind (default unknown))
  (slot destination (default ""))
  (slot relation (default ""))
  (slot value (default ""))
  (slot qualifier (default "")))
```

Para el segundo objetivo del ejemplo conductor, la representación es:

```
(gpsr-goal
```

```
(step 2)
(name find)
(target "chocolate_jello")
(kind object)
(destination "")
(relation "")
(value "")
(qualifier ""))
```

Los espacios de los argumentos se convierten en guiones bajos para construir los identificadores de ejecución. Los valores de cadena se escapan antes de insertarse en CLIPS, mientras que el nombre y el tipo se representan como símbolos.

La clasificación de entidades utiliza la secuencia completa. Los destinatarios de entrega, los participantes de guía, seguimiento y saludo, los atributos visuales y la capitalización de los nombres aportan indicios para interpretar una búsqueda como búsqueda de persona. El campo **kind** del hecho se calcula mediante estas reglas de contexto; no se copia directamente del argumento homónimo de la predicción. Por ejemplo, **go** y **place** se clasifican como ubicación, **take** y **deliver** como objeto, y las operaciones de información como **info**. Esta clasificación cumple una función operativa en el adaptador y no redefine el vocabulario semántico.

ClipsGoalFactBuilder conserva los destinos indicados por **to**, **at**, **on** o **in**, en ese orden de prioridad. También admite el segundo argumento posicional para entrega, guía, seguimiento y colocación. La relación principal selecciona el primer campo disponible entre destino, gesto, postura, vestimenta y propiedad. El campo **qualifier** conserva el primer atributo visual, con prioridad de gesto, postura, vestimenta y propiedad, en la forma **clave:valor**. La representación no codifica una conjunción de varios calificadores visuales.

Además de la normalización léxica, el adaptador aplica una reparación específica cuando recibe una navegación cuyo argumento corresponde a una persona. Ese objetivo se transforma en búsqueda. Si inmediatamente después aparece un seguimiento de la misma persona con destino, se adelanta el destino como navegación, se añade la búsqueda y se conserva el seguimiento sin destino. Esta operación modifica la interpretación de la secuencia y debe registrarse como posprocesamiento, sin atribuir la corrección al modelo.

Para mantener la notación del capítulo anterior, la salida del intérprete conserva el

símbolo $G_{\text{pred}}^{\text{Qwen}}$. Se denomina Adaptar a la transformación del adaptador, que comprende el análisis, las reparaciones, la clasificación y la serialización. El recorrido implementado queda expresado como:

$$P^{\text{Qwen-CLIPS}} = \text{CLIPS}\left(\text{Adaptar}\left(G_{\text{pred}}^{\text{Qwen}}\right)\right). \quad (5.1)$$

La calidad de la interpretación se calcula sobre la predicción anterior a esta transformación. La evaluación del sistema completo considera también los hechos y el plan que resultan de ella.

La auditoría fuera de línea utiliza `goal_to_clips_fact` para construir el mismo tipo de hecho, pero emplea el análisis del esquema semántico y no reproduce todas las heurísticas y reparaciones del nodo ROS 2. En consecuencia, compartir la plantilla `gpsr-goal` no garantiza equivalencia entre ambos recorridos. La comprobación integrada debe conservar la transformación que efectivamente recibió CLIPS.

5.4.2. Representación del estado simbólico

La planificación comienza con un estado que representa ubicación desconocida y ausencia de un objeto sostenido:

```
(current-state (location "unknown") (holding "nothing"))
```

Las reglas actualizan este estado al añadir acciones al plan. Los valores representan efectos previstos, no confirmaciones físicas. Por ejemplo, la incorporación de una adquisición modifica el objeto esperado en posesión del robot. La colocación exige que coincida con el objeto solicitado; las reglas de entrega no imponen esa misma igualdad como precondition general. La validación de la secuencia de objetivos y las condiciones de expansión son, por tanto, controles complementarios.

Otros hechos conservan referencias a personas, el último contexto de interacción, análisis visuales y el estado del foco. Las referencias `guest_N` y `customer_N` permiten reutilizar una entidad durante distintas subtarefas. Los objetos se identifican mediante nombres normalizados e índices, como `chocolate_jello_1`.

La referencia `guest_last` se utiliza para una persona capturada por `find_person`, mientras que `customer_N` identifica una búsqueda por atributo realizada con `find_pose`. Una persona nombrada puede requerir búsqueda anónima, solicitud del nombre y escucha mediante `guest_name_last`. La asociación efectiva de estos identificadores con detecciones y respuestas corresponde al ejecutor.

El modo de manipulación se representa mediante `manipulation-mode`. Su valor predeterminado está habilitado y puede cambiarse mediante `set-manipulation-enabled`. Cuando se deshabilita, una adquisición produce una explicación hablada y un hecho `manipulation-unavailable`; las operaciones dependientes generan el aviso correspondiente. Esta salida informa una capacidad no disponible y no acredita la realización de la tarea de manipulación.

5.4.3. Reglas de expansión

Las reglas se agrupan en `goals_planning.clp`. Una regla de procesamiento consume un objetivo únicamente cuando no existe otro pendiente con un paso menor. Después de expandirlo, retira el hecho correspondiente y añade las acciones de la subtarea.

La navegación a una ubicación distinta de la registrada produce una solicitud `go_to` y una confirmación hablada. Si ambas ubicaciones coinciden, el objetivo se consume sin repetir el desplazamiento. La regla siguiente muestra esta decisión y la actualización del estado:

```
(defrule process-go
  (declare (salience 200))
  (planning-ready)
  ?g <- (gpsr-goal (step ?step) (name go) (target ?dest))
  (not (gpsr-goal (step ?prev&:(< ?prev ?step))))
  ?st <- (current-state (location ?loc) (holding ?held))
  =>
  (retract ?g)
  (if (neq ?loc ?dest) then
    (append-say (str-cat "I will navigate to " ?dest))
    (append-action go_to ?dest)
    (append-action say (str-cat "I have arrived at " ?dest))
    (retract ?st)
    (assert (current-state (location ?dest) (holding ?held)))))
```

La búsqueda de objetos genera `find_object` o una operación de análisis cuando se utiliza una propiedad admitida. Las búsquedas de personas recurren a `find_person` o `find_pose` y conservan el contexto necesario para la interacción posterior.

Con la manipulación habilitada, la adquisición utiliza `take`; la entrega incorpora aproximación y `give`; la colocación puede incluir navegación al destino, `find_space` y `drop`. La selección concreta depende de los argumentos y del estado simbólico. Las acciones de diálogo y de control del foco complementan estas operaciones.

Para la colocación se distinguen puntos de navegación y superficies genéricas. Un destino como `storage_rack` puede añadir navegación, mientras que `table`, `shelf` o

`cabinet` se tratan como soportes en el lugar actual. La función siguiente incorpora la búsqueda de espacio y la liberación del objeto; el suelo constituye una excepción que utiliza directamente el argumento `floor`:

```
(deffunction append-drop-sequence (?obj ?destination)
  (bind ?surface (drop-surface ?destination))
  (if (eq ?surface "floor") then
    (append-action drop "floor")
    else
    (append-action find_space ?surface)
    (append-action drop (object-id ?obj))))
```

En este fragmento, `drop` identifica la acción del ejecutor que materializa el objetivo semántico `place`. La búsqueda de espacio selecciona una clase de soporte, como mesa, estante o contenedor, y la liberación utiliza el identificador del objeto.

La guía genera una instrucción hablada, navegación al destino y cierre de la interacción. El seguimiento recupera o busca a la persona, incorpora aproximación cuando corresponde y emite mediante `say` las claves `instruct_follow_only` o `instruct_follow_to_<destino>`. Esta regla no emite una acción explícita de seguimiento ni actualiza la ubicación al destino. Su expansión representa el protocolo de interacción; la realización física del seguimiento depende del tratamiento de esas claves en el ejecutor y debe verificarse por separado.

Las propiedades de tamaño se traducen a consultas de análisis de objetos grandes o pequeños. La implementación también utiliza el tamaño como aproximación para las solicitudes de mayor o menor peso y menor grosor. Esta correspondencia permite producir una consulta visual, pero no constituye una medición de peso o grosor ni demuestra la satisfacción de esas propiedades. Estos casos requieren una interpretación diferenciada en la evaluación.

El conteo genera `analyze_objects`, incorpora retorno al punto de instrucción cuando corresponde y emite la respuesta `count_category`. Para guardar un nombre se solicita y escucha `guest_name_last`; para atributos de persona se utilizan operaciones de análisis o búsqueda por postura. La información adquirida se comunica mediante las claves de respuesta de `tell`. La respuesta a preguntas combina `listen` y `say` con la clave `answer_question`, y el saludo utiliza la persona referenciada y la clave `greet`.

5.4.4. Generación de acciones

Las reglas construyen hechos intermedios **action-step**, que posteriormente se convierten en **ros-message**. Los anuncios generales utilizan numeración desde 1 y las acciones operativas desde 1001. Al ordenar la salida, se presenta primero el anuncio de las tareas y después las operaciones del plan.

En el ejemplo conductor, la secuencia operativa central puede representarse como sigue. Se omiten aquí los anuncios y confirmaciones para mostrar la relación con los cuatro objetivos:

```
go_to      bed
find_object chocolate_jello_1
take       chocolate_jello_1
go_to      instruction_point
approach    host_1
give       chocolate_jello_1
```

El objetivo de entrega al operador se expande en retorno al punto de instrucción, aproximación y entrega física. Esta secuencia es una ilustración de la expansión; el plan completo incluye también las acciones de comunicación y finalización establecidas por las reglas.

5.4.5. Condición de finalización

Cuando no quedan objetivos pendientes, las reglas desactivan el foco si continúa activo, añaden el retorno a **instruction_point** cuando corresponde y generan el mensaje de tarea completada. La construcción termina con la nota **GPSR_DONE**.

La finalización se distribuye en tres reglas. Primero, **add-final-return** agrega el cierre cuando no quedan objetivos. Después, **serialize-action-step** transforma las acciones en mensajes únicamente cuando existe **final-steps-added**. Finalmente, **goals-finish** emite la nota de cierre cuando tampoco quedan hechos **action-step** por serializar. Las prioridades de estas reglas son 20, 5 y 1, respectivamente.

El componente **PlanExtractor** ordena los hechos **ros-message** por su paso y construye una cadena con una instrucción por línea. Las notas **GPSR_START** y **GPSR_DONE** delimitan el plan publicado. La salida se añade también al archivo **plan_execution.log**.

Estas notas corresponden a la generación del plan. Su presencia no constituye una confirmación de ejecución del robot. La comprobación funcional revisa asimismo que no queden objetivos pendientes y que no aparezca la respuesta de objetivo no soportado.

La regla de objetivo no soportado consume el hecho y añade una explicación hablada. Por esta razón, la ausencia de objetivos pendientes y la marca de cierre pueden coexistir con una incapacidad de expansión. El diagnóstico necesita examinar también el contenido del plan, incluidas las respuestas de manipulación no disponible.

5.5. Integración Qwen–CLIPS con Justina

5.5.1. Componentes ROS 2

La integración utiliza un nodo semántico, un adaptador de objetivos y el nodo que administra CLIPS. El nodo `qwen_semantic_node.py` actúa como cliente del servidor de inferencia y entrega la representación con `goals`. El adaptador coordina la inserción de hechos y el inicio de la planificación.

El nodo `clips_node.py` expone los servicios del motor y publica el plan. El acceso al entorno se serializa mediante una cola de tareas procesada por un único trabajador, evitando que los callbacks manipulen concurrentemente el mismo entorno CLIPS. El componente `clips_engine.py` encapsula la interacción con `clips.Environment`.

El lanzamiento de integración agrupa los componentes bajo el espacio de nombres `/planning`. La selección del modo de objetivos utiliza el parámetro `plan_name=goals`, que permite cargar `goals_planning.clp`. El formato de respuesta del nodo semántico se configura como `goals` para entregar al adaptador el contrato esperado.

El adaptador utiliza un nodo auxiliar de clientes y un `SingleThreadedExecutor` en un hilo dedicado. Mientras el callback del servicio de conversión espera la inserción de hechos o el inicio de planificación, ese ejecutor procesa las respuestas de los servicios dependientes. Así se evita que ambas operaciones dependan del mismo callback bloqueado.

5.5.2. Flujo de comunicación

La orden ingresa mediante el servicio del parser semántico. El nodo envía la solicitud al servidor Qwen y recibe la predicción validada. La lista de objetivos se entrega después

al servicio de conversión, que coordina dos operaciones: reiniciar e insertar el lote de hechos, y solicitar el inicio de planificación.

En el despliegue bajo `/planning`, las interfaces principales son las siguientes:

```
/planning/semantic_parser/semantic_parser
/planning/goals_to_clips/convert
/planning/clips/assert_facts_batch
/planning/clips/start_planning
/planning/clips/results
```

El servicio de conversión utiliza `clips_interfaces/srv/String`: recibe el JSON en `request.data` y devuelve el estado en `response.data`. La inserción por lote utiliza el tipo `clips_interfaces/srv/StringArray`. El inicio de planificación se solicita mediante `std_srvs/srv/Trigger`. El hecho (`start (name action-planning)`) activa la inicialización de las reglas y la expansión de los objetivos cargados.

El plan completo se publica como un mensaje `std_msgs/msg/String`. La aceptación de la solicitud de conversión indica que se completó la coordinación de servicios; la obtención y el contenido del plan se verifican sobre el resultado publicado. El script `qwen_to_plan.py` permite recorrer este flujo con una sola orden y mostrar la respuesta semántica, el estado de conversión y el plan.

El adaptador espera hasta tres segundos por la disponibilidad de cada servicio dependiente y hasta cinco segundos por su respuesta. Comprueba `successful` tras la inserción y `success` tras el inicio de planificación. Solo devuelve `OK` si ambas operaciones responden favorablemente; en caso de fallo devuelve una descripción de error. Una lista vacía se rechaza antes del envío. Estos límites corresponden a cada operación y no equivalen a un tiempo máximo global del recorrido completo.

Los nombres de conversión, inserción e inicio se declaran como rutas absolutas bajo `/planning`. El adaptador solicita el reinicio y la inserción mediante el servicio por lote; el manejo efectivo de la memoria de CLIPS pertenece al nodo que atiende ese servicio.

5.5.3. Comunicación con el ejecutor

El componente `plan_manager_node` recibe el plan, extrae las líneas que representan pasos y conserva las instrucciones ordenadas. Cada solicitud a `/request_instruction` obtiene la siguiente instrucción y la publica en `/instruction_msg`. Cuando no existe un plan asignado, la respuesta indica `NoPlanAssigned`.

La suscripción del gestor utiliza `/clips/results`; cuando el planificador publica bajo `/planning`, el lanzamiento debe resolver esta correspondencia mediante el remapeo pertinente. La configuración de nombres forma parte de la integración y permite que el consumidor reciba el mismo contrato de salida.

La entrega secuencial separa la construcción del plan de su consumo. La confirmación de éxito de una acción, el tratamiento de fallos y la decisión de continuar corresponden a las interfaces del sistema de ejecución. La publicación de la siguiente instrucción, por sí sola, no acredita que la anterior se haya realizado físicamente.

Durante el desarrollo se utilizaron 48 comandos como prueba de funcionamiento. Esta comprobación se considera parte de la verificación del sistema y se distingue del conjunto experimental del capítulo 6.

5.6. Implementación del planificador GPT

La preparación del componente directo comprende la interfaz de solicitud, el prompt, el contrato de salida y las comprobaciones que anteceden al consumo del plan. Estos elementos concretan el diseño de la Sección 4.5 y mantienen separados los objetivos declarados y las acciones generadas.

5.6.1. Interfaz con la API

La interfaz se especifica mediante una solicitud a la API de Responses. El mensaje de sistema contiene las instrucciones del planificador y el mensaje de usuario contiene la orden. La solicitud incorpora también el esquema de respuesta para delimitar el formato de los objetivos y del plan.

La configuración separa el identificador del modelo del contenido del prompt y del esquema. Para la evaluación se fijarán estas versiones y los parámetros de generación. La respuesta deberá asociarse con la solicitud que la originó, conservando tanto su contenido como la información de uso disponible y el tiempo observado por el cliente.

El procesamiento distinguirá las respuestas del modelo de los errores de servicio, interrupciones y límites de espera. Estos últimos se registrarán como incidencias de la solicitud y no como decisiones de aclaración o rechazo de la tarea.

5.6.2. System prompt

Se elaboró un prompt en inglés con el dominio de entidades, las formas canónicas de los objetivos y las convenciones de expansión. El contenido especifica cómo resolver referencias y cómo distinguir personas, objetos, ubicaciones e información. También establece las dependencias entre búsqueda, adquisición, transporte e interacción.

Las instrucciones operativas definen el estado esperado durante la generación: ubicación, objeto sostenido, foco de interacción y referencias a personas. Se incluyen convenciones de identificación, numeración de pasos y finalización del plan. El vocabulario de salida se ajusta al contrato de la investigación.

Para evitar que el contexto cambie inadvertidamente entre solicitudes, el prompt se conserva como un artefacto separado de la orden. Su versión deberá permanecer fija durante la comparación. Las instrucciones operativas permiten examinar la generación de GPT bajo las mismas convenciones de tarea que utiliza el expansor simbólico.

5.6.3. JSON Schema y Structured Outputs

El contrato prevé utilizar JSON Schema mediante Structured Outputs para restringir la estructura de la respuesta. Los campos principales se resumen en la Tabla 5.2. El plan contiene pasos con un número de orden, el identificador del robot, el nombre de la acción y sus parámetros.

Tabla 5.2: Campos del contrato de respuesta del planificador directo.

Campo	Función
<code>status</code>	Tipo de respuesta a la instrucción.
<code>message</code>	Pregunta de aclaración o explicación del rechazo.
<code>goals</code>	Secuencia de objetivos semánticos.
<code>plan</code>	Pasos de acciones compatibles con el contrato del ejecutor.
<code>start_note</code>	Marca de inicio del plan ejecutable.
<code>end_note</code>	Marca de finalización de la generación.

El esquema y los criterios de aceptación cumplen funciones distintas. La estructura permite verificar campos, tipos y nombres de acciones; las relaciones entre estado, objetivos y pasos requieren comprobaciones adicionales. Un objeto JSON bien formado puede contener un destinatario incorrecto o una secuencia que incumpla las dependencias de la tarea.

5.6.4. Generación de objetivos y planes

Para una instrucción ejecutable, el contrato solicita que el modelo produzca $G_{\text{pred}}^{\text{GPT}}$ y P^{GPT} en la misma respuesta. Los objetivos permiten comparar la interpretación con G_{ref} ; los pasos permiten examinar la realización de la tarea y su compatibilidad con Justina.

El ejemplo conductor requiere identificar el objeto y su ubicación, adquirirlo y entregarlo al operador. La especificación directa debe conducir a acciones con los mismos tipos de identificadores utilizados en la Sección 5.4, aunque la producción del plan ocurra dentro del modelo.

La salida de GPT no se introduce en CLIPS. Su consumo requiere validar el contrato y transformar la representación de los pasos al formato que recibe el ejecutor. Esa transformación de formato debe distinguirse de una modificación del contenido operativo del plan.

5.6.5. Tratamiento de aclaraciones y rechazos

El contrato utiliza `clarification` cuando falta información necesaria que puede solicitarse mediante una pregunta breve, y `rejected` cuando la tarea excede el dominio o las capacidades admitidas. En ambos casos, los campos de objetivos y plan se devuelven vacíos y no se incluyen las marcas de un plan ejecutable.

El consumidor debe utilizar el estado para decidir si existe un plan susceptible de validación y ejecución. Una pregunta de aclaración no se interpreta como una acción adicional del plan. La evaluación de estas respuestas se separa de la generación de acciones para las instrucciones resolubles del dominio común.

5.6.6. Validación determinista

La integración prevé una validación posterior a la generación para comprobar el estado de respuesta, la estructura de los pasos y su compatibilidad con el repertorio de acciones. Para los planes ejecutables se verifican también el orden, los parámetros y las marcas de inicio y cierre. Las respuestas de aclaración y rechazo deben mantener vacíos los campos operativos.

La validación de objetivos puede utilizar el contrato semántico común. La consistencia de las acciones requiere controles específicos sobre los efectos esperados y las

dependencias del plan. Esta comprobación no delega la planificación a CLIPS ni convierte la arquitectura directa en una cadena GPT-CLIPS.

Se conservará el resultado original junto con las incidencias del validador. Las comprobaciones implementadas deberán distinguirse de las condiciones que solo se solicitan en el prompt, para identificar qué restricciones se comprueban efectivamente fuera del modelo.

5.7. Reproducibilidad

5.7.1. Versiones de modelos

La identificación de Qwen comprende la versión del modelo base, el tokenizador y el adaptador seleccionado. El registro deberá permitir relacionar estos artefactos con el conjunto de datos y la ejecución de entrenamiento. La conservación de un checkpoint aislado sin su configuración no permite reconstruir toda la condición experimental.

Para GPT se registrarán el identificador del modelo utilizado en la solicitud, la fecha de ejecución y las versiones del prompt y del esquema. La comparación se realizará sin cambiar estas condiciones durante el procesamiento del conjunto de prueba.

5.7.2. Configuraciones experimentales

La configuración de cada ejecución incluirá los catálogos, los parámetros de normalización e inferencia, la base de reglas, el modo de manipulación y el despliegue de los componentes. El mismo criterio se aplica a los validadores y a los recorridos de conversión de objetivos.

Para la inferencia se conserva también la distribución de dispositivos, el número de hilos de CPU y la opción TF32. El módulo configura por defecto los hilos de cómputo a partir del número de CPU disponibles y limita los hilos de interoperación a ocho. Cuando CUDA está disponible, habilita TF32 salvo que se indique `--no-tf32`. Estas opciones pueden afectar los tiempos y forman parte de la condición evaluada.

El dataset y las particiones se vincularán con sus tamaños, semillas y versiones. Las pruebas de funcionamiento se mantendrán identificadas como parte del desarrollo, mientras que el conjunto de comparación se gestionará de acuerdo con las condiciones del capítulo 6.

5.7.3. Registro de ejecuciones

El sistema simbólico conserva el plan publicado en `plan_execution.log`. Para la evaluación se completará este registro con el identificador de la orden, la entrada efectiva, los objetivos originales, las transformaciones del adaptador, el plan y las incidencias de cada etapa.

El módulo de inferencia incorpora **ResourceTimer** para registrar tiempo transcurrido, tiempo de CPU del proceso, memoria RAM máxima del proceso y memoria de GPU asignada y reservada, junto con sus máximos. Las mediciones temporales sincronizan CUDA en los límites del bloque observado. El porcentaje de CPU expresa la razón entre tiempo de CPU y tiempo transcurrido, por lo que puede superar el 100 % si intervienen varios hilos. El máximo de RAM corresponde a la vida del proceso, no al incremento causado únicamente por una solicitud.

Estos diagnósticos se imprimen en la ejecución del módulo y se distinguen de `elapsed_ms`, que el servidor devuelve para la traducción. El servidor no incorpora automáticamente todas las métricas de **ResourceTimer** a cada respuesta HTTP. La latencia completa de Qwen-CLIPS requiere además medir la comunicación, la conversión de objetivos y la expansión simbólica con límites temporales explícitos.

En la arquitectura directa se conservarán la respuesta original y el resultado de las comprobaciones, junto con la latencia y la información de uso de tokens disponible. Los errores de solicitud y las salidas inválidas se incluirán en el registro para evitar que la evaluación considere únicamente los casos que produjeron un plan aceptado.

Esta información permite relacionar un resultado con la configuración que lo produjo y separar interpretación, conversión, planificación y ejecución. El capítulo siguiente establece el procedimiento experimental con el que se evaluarán esas etapas.

Capítulo 6

Validación y experimentación

6.1. Diseño experimental

6.1.1. Conjunto de prueba

6.1.2. Procedimiento experimental

6.1.3. Repeticiones y control de variabilidad

6.2. Condiciones experimentales

6.2.1. Arquitectura Qwen–CLIPS

6.2.2. Arquitectura GPT directo

6.2.3. Condición de control con objetivos de referencia y CLIPS

6.3. Evaluación de la interpretación semántica

6.3.1. Exactitud de objetivos

6.3.2. Validez estructural

6.3.3. Exactitud de entidades y parámetros

6.3.4. Dependencias entre objetivos

6.4. Evaluación de la planificación

6.4.1. Acciones válidas

6.4.2. Consistencia del plan

Capítulo 7

Resultados

7.1. Resultados de la especialización de Qwen

7.1.1. Comportamiento durante entrenamiento

7.1.2. Evaluación del modelo seleccionado

7.2. Resultados de interpretación semántica

7.2.1. Qwen

7.2.2. GPT

7.2.3. Comparación

7.3. Resultados de planificación

7.3.1. Qwen–CLIPS

7.3.2. GPT directo

7.3.3. Condición de control CLIPS

7.3.4. Comparación

7.4. Análisis de errores

7.4.1. Errores de interpretación

7.4.2. Errores de planificación

7.4.3. Casos representativos

Capítulo 8

Discusión

8.1. Interpretación de los resultados respecto a la hipótesis

8.2. Interpretación semántica

8.2.1. Especialización de Qwen

8.2.2. Capacidad de GPT

8.2.3. Efecto de la variabilidad lingüística

8.3. Planificación de acciones

8.3.1. Expansión determinista mediante CLIPS

8.3.2. Planificación generativa mediante GPT

8.3.3. Separación entre errores de interpretación y planificación

8.4. Comparación Qwen–CLIPS frente a GPT directo

8.4.1. Exactitud

8.4.2. Consistencia

8.4.3. Latencia

Capítulo 9

Conclusiones y trabajo futuro

9.1. Conclusiones principales

9.2. Cumplimiento de los objetivos

9.3. Evaluación de la hipótesis

9.4. Contribuciones de la investigación

9.4.1. Representación canónica de órdenes

9.4.2. Arquitectura Qwen–CLIPS

9.4.3. Comparación con planificación directa mediante GPT

9.4.4. Metodología para separar errores de interpretación y planificación

9.5. Trabajo futuro

Capítulo 10

Conclusiones y Trabajo Futuro

10.1. Conclusiones Principales

10.2. Propuestas de Trabajo Futuro

- Fine-tuning de un LLM de código abierto para dominio específico
- Mejora de los mecanismos de seguridad y verificación
- Exploración de arquitecturas de integración más profundas

Referencias

- [1] R. R. Murphy, *Introduction to AI Robotics*. MIT Press, 2nd ed., 2019.
- [2] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*. Pearson, 4th ed., 2020.
- [3] J. C. Giarratano and G. D. Riley, *Expert Systems: Principles and Programming*. Thomson Course Technology, 4th ed., 2005.
- [4] R. A. Brooks, “A robust layered control system for a mobile robot,” *IEEE Journal of Robotics and Automation*, vol. 2, no. 1, pp. 14–23, 1986.
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [6] L. Shi, X. Chen, Y. Hu, J. Li, H. Zhang, and H. Zhao, “Chatgpt for robotics: Design principles and model abilities,” *Microsoft Research*, 2023.
- [7] NASA Johnson Space Center, “Clips reference manual,” tech. rep., NASA, 1991.
- [8] B. Pell *et al.*, “An autonomous spacecraft agent prototype,” in *First International Conference on Autonomous Agents*, pp. 253–261, 1997.
- [9] J. Savage and otros, “Virbot: A modular architecture for service robots with hybrid planning,” in *RoboCup Symposium*, 2024.
- [10] L. Cheng, Y. Mao, and T. Ward, “Editorial: Neural network models in autonomous robotics,” *Frontiers in Neurorobotics*, 2025.
- [11] Alibaba Cloud Qwen Team, “Qwen official github repository,” 2026.
- [12] OpenAI, “Openai developer documentation and model guides,” 2026.
- [13] N. Houlsby, A. Giurciu, S. Jastrzebski, B. Morrone, Q. De Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, “Parameter-efficient transfer learning for

- NLP,” in *Proceedings of the 36th International Conference on Machine Learning*, vol. 97 of *Proceedings of Machine Learning Research*, pp. 2790–2799, PMLR, 2019.
- [14] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “LoRA: Low-rank adaptation of large language models,” in *International Conference on Learning Representations*, 2022.
- [15] T. Detrmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient finetuning of quantized LLMs,” in *Advances in Neural Information Processing Systems*, vol. 36, pp. 10088–10115, 2023.
- [16] A. d’Avila Garcez, T. R. Besold, L. De Raedt, P. Földiák, P. Hitzler, T. Icard, K.-U. Kühnberger, L. C. Lamb, R. Miikkulainen, and D. L. Silver, “Neural-symbolic learning and reasoning: Contributions and challenges,” in *Knowledge Representation and Reasoning: Integrating Symbolic and Neural Approaches: Papers from the 2015 AAAI Spring Symposium*, pp. 18–21, AAAI Press, 2015.
- [17] T. R. Besold, A. d’Avila Garcez, S. Bader, H. Bowman, P. Domingos, P. Hitzler, K.-U. Kuehnberger, L. C. Lamb, D. Lowd, P. M. V. Lima, L. de Penning, G. Pinkas, H. Poon, and G. Zaverucha, “Neural-symbolic learning and reasoning: A survey and interpretation,” *arXiv preprint arXiv:1711.03902*, 2017.
- [18] B. Liu, Y. Jiang, X. Zhang, Q. Liu, S. Zhang, J. Biswas, and P. Stone, “LLM+P: Empowering large language models with optimal planning proficiency,” *arXiv preprint arXiv:2304.11477*, 2023.
- [19] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, *et al.*, “Qwen2.5 Technical Report,” *arXiv preprint arXiv:2412.15115*, 2024.
- [20] J. Hart, A. Moriarty, K. Pasternak, J. Kummert, M. Leonetti, L. Contre-ras, L. Ruegmer, A. Mitzutani, T. Ribeiro, A. Golding, T. Kang, and F. Pimentel, “RoboCup@Home 2026: Rules and Regulations.” <https://github.com/RoboCupAtHome/RuleBook/releases/tag/2026.2>, 2026. RoboCup@Home Technical Committee.
- [21] RoboCup@Home, “CommandGenerator: Command Generator for RoboCup@Home.” <https://github.com/RoboCupAtHome/CommandGenerator>, 2026. Repositorio oficial de RoboCup@Home.

-
- [22] RoboCup@Home, “CompetitionTemplate.” <https://github.com/RoboCupAtHome/CompetitionTemplate>, 2026. Repositorio oficial de RoboCup@Home.